

Garuda 6-Step Sensorless BLDC ESC

Architecture, control algorithms, timing & the complete signal-processing mathematics

Target MCU: dsPIC33AK512MC510 (MC510 / "AK512") — 100 MHz FCY, 400 MHz PWM clock

Board: MCLV-48V-300W + EV67N21A DIM · 3 mΩ shunt, ×24.95 amps

Topology: 3-phase 6-step trapezoidal, sensorless BEMF zero-cross commutation

Scheme: ADC-digital-comparator HWZC + software ZC, sector-PI PLL, RC filter-lag compensation

Reference build: MOTOR_PROFILE_6 (VEX 14 mm 4000 KV, 6 PP) on branch ak512-port

This document is generated from a full read of the firmware source tree (dspic33AKESC/). Every formula, constant, register and API in this note is taken directly from the code; file/line citations use the form `file.c:line`. Pure-6-step paths are documented as live; FOC paths are noted where they share infrastructure.

Contents

1. System Overview & Architecture

Clock tree & time bases
Interrupt / task structure
Data model

2. Control State Machine & Startup

States & transitions
Align → ramp → morph → closed-loop
Startup strategies
Fault handling & recovery

3. PWM Generation & 6-Step Commutation

The PWM peripheral
Override-based 6-step drive
ADC triggers

4. Zero-Crossing Detection

Two detection paths
Neutral / threshold mathematics
Sample point in the PWM cycle
Falling-SW per-polarity detection
Validation & rejection logic

5. Commutation Timing: PLL, Sector-PI & IIR

The autonomous-timer PLL
The sector PI controller
The IIR period filter

6. RC Filter-Lag Compensation — Full Mathematics

The physical lag
The threshold-bias formula
Per-polarity asymmetry
The amplitude source

7. Timing Advance

8. ABS_FLOOR False-Lock Guard & the λ Math

9. Protection & Auxiliary Control

The overcurrent ladder
OC auto-zero bias
Bus-voltage tiers
Outer speed PI (parked)

10. GSP Host Protocol & Parameter System

Wire protocol & CRC
Command set
Parameter & descriptor tables
Telemetry snapshot

11. Consolidated Mathematics Reference

12. Appendix: Feature Flags & Glossary

1 System Overview & Architecture

The Garuda AKESC firmware drives a 3-phase brushless motor with no position sensor. Rotor position is inferred from the back-EMF (BEMF) of whichever phase is momentarily not driven. The firmware energises two of the three phases (one high-side PWM, one low-side ground return) and leaves the third floating; the floating phase's terminal voltage crosses the motor's virtual neutral exactly half-way between commutations, and that *zero-crossing* (ZC) is the timing reference for the entire control loop.

Two ZC detectors run cooperatively: a low-speed **software ZC** that thresholds the floating-phase ADC sample inside the control ISR, and a high-speed **hardware ZC (HWZC)** built on the on-chip ADC digital comparators plus SCCP capture/timers. A **sector-PI PLL** tracks the commutation period in 10 ns resolution and schedules the next commutation autonomously, with timing advance, an RC-filter-lag compensator, and an absolute no-load floor guarding against false harmonic lock.

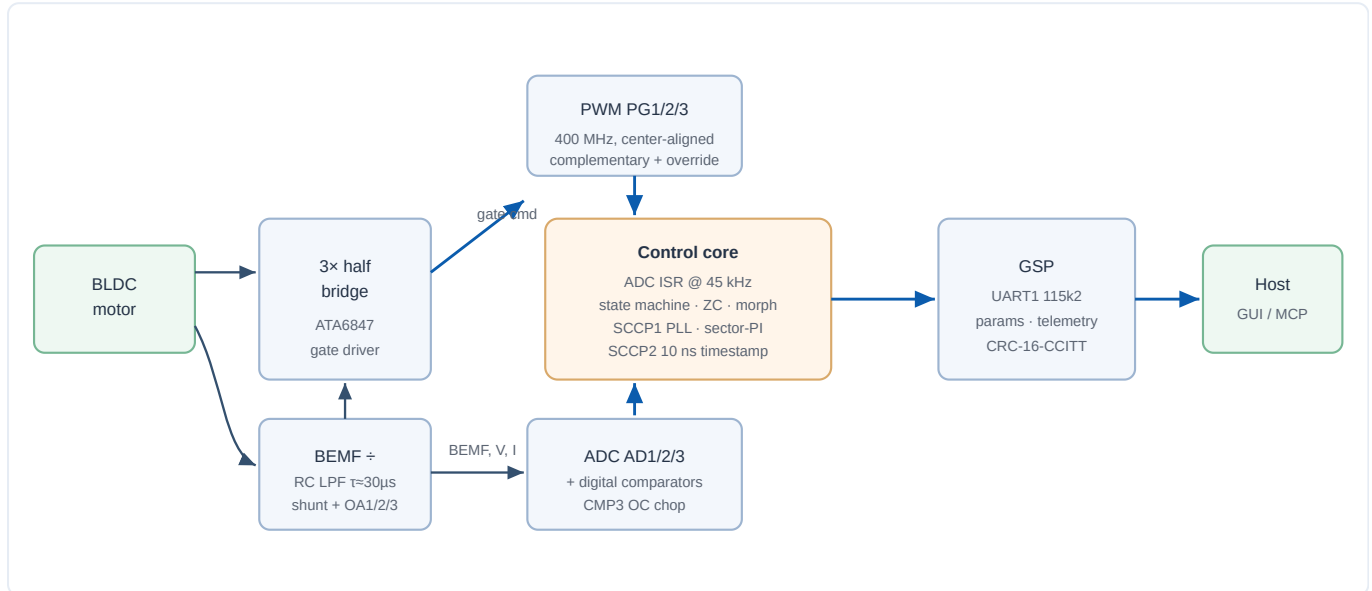


Figure 1. Top-level signal flow. The control core (orange) runs entirely on-chip; the host link is optional and never in the commutation path.

1.1 Clock tree & time bases

The oscillator (clock.c:99) starts from the 8 MHz FRC and runs PLL1 with M=100, N1=1, N2=4, N3=1:

$$F_{VCO} = 8 \text{ MHz} \times 100 / 1 = 800 \text{ MHz} \rightarrow F_{OSC} = 8 \times 100 / (1 \cdot 4 \cdot 1) = 200 \text{ MHz} \rightarrow F_{CY} = 100 \text{ MHz}$$

clock.c

Per-peripheral generators: CLK5 = 800/2 = **400 MHz** (PWM, PWM_CLOCK_MHZ=400); CLK6 = 100 MHz (ADC); CLK7 = 400 MHz (DAC/CMP). Every SCCP timer uses CLKSEL=0 = F_{CY}, so SCCP_CLOCK_HZ = 100 000 000 and **1 high-resolution (HR) tick = 10 ns**. The four key time bases:

Time base	Rate / period	Source	Role
PWM & control ISR	PWMFREQUENCY_HZ = 45 000 → 22.22 μs	PG1 center-aligned	One ADC conversion set + one control-ISR pass per period
SCCP2 (HR timestamp)	100 MHz, 10 ns, wraps ≈ 42.9 s	free-running	Stamps every ZC; the period/eRPM measurement clock
SCCP1 (one-shot)	100 MHz, programmable	software-triggered	Schedules blanking expiry & the next commutation
SCCP3 (periodic)	1 MHz (100-tick period)	periodic trigger	Free-running 1 MHz ADC trigger for the HWZC comparator channel
Timer1	1:8 prescale → 100 μs (10 kHz)	F _{CY} /8	Arm/align/ramp/recovery counters, buttons, 1 ms systemTick

The PWM master period register is

$$LOOPTIME_{TCY} = (LOOPTIME_{\mu s} \times 8 \times PWM_CLOCK_MHZ) - 16 = (22.222 \times 8 \times 400) - 16 \approx 71\,095 \text{ counts}$$

garuda_calc_params.h:31

The $\times 8$ = ($\times 2$ center-aligned up/down) $\times$ ($\times 4$ high-resolution divider). Duty is in the same counts, so $duty / LOOPTIME_{TCY}$ is the duty fraction D . Two structural minimums fall out of the 300 ns deadline:

$$DEADTIME_COUNTS = (DEADTIME_NS / 1000) \times 16 \times 400 = 0.3 \times 16 \times 400 = 1920 \quad | \quad MIN_DUTY = 2 \times DEADTIME_COUNTS = 3840$$

garuda_calc_params.h:36–39

MIN_DUTY (= 2× deadline, ≈ 5.4 % duty) is the floor below which there is no usable on-time after the module swallows deadline on both edges. This single fact sets the ≈ 10 k eRPM closed-loop idle wall referenced throughout the project.

1.2 Interrupt / task structure

All control runs in interrupts; `main()` does only background housekeeping (buttons, GSP service, heartbeat watchdog, CPU-load estimate, intent-flag → state transitions). No control math executes in `main`.

Context	Priority	Rate	Owens
Comparator ZC ISRs + SCCP1_CCT1	7 (top)	event / sub-μs	HWZC state machine: blanking, capture, sector-PI, commutation
ADC ISR GARUDA_ADC_INTERRUPT	6–7	45 kHz	Fault checks, threshold update, software ZC, the whole morph blend, the 6-step switch
Timer1_T1Interrupt	5	10 kHz	Arm/align/ramp/recovery timing, button debounce, systemTick
SCCP4 input-capture / DShot DMA	4 / 1	RX edges	Throttle pulse / DShot decode (not in the motor loop)
<code>main()</code> while(1)	—	background	GSP, buttons, heartbeat, CPU-load — no control

Priority-7 preempts the ADC ISR, so once HWZC is enabled the hardware ZC edge is serviced with bounded latency even if the 45 kHz ADC ISR is mid-pass. The two paths share the `GARUDA_DATA_T` state and use a *seqlock* (`hwzc.writeSeq`) on the 32-bit `stepPeriodHR` so a lower-priority reader never tears it.

1.3 Data model

One `GARUDA_DATA_T` aggregate (`garuda_types.h:633`) holds the whole control state. Sub-structures used throughout this note:

- `BEMF_STATE_T bmf` — `bmfRaw` (u16, floating-phase ADC), `zcThreshold` (u16, the neutral threshold), `zcAmpForFilterComp` (u16, slow-IIR measured BEMF amplitude), `bmfDevPeak` (u16, running peak |float-neutral|), `bmfSampleValid` (bool).
- `HWZC_STATE_T hwzc` — `phase` (IDLE/BLANKING/WATCHING/COMM_PENDING), `stepPeriodHR` (u32, HR ticks, seqlock-protected), `timerPeriod` (u32, PI-controlled commutation period), `integratorF` (float PI integrator), `lastZcStamp` / `lastCommStamp` / `lastCaptureHR` (u32 HR), `goodZcCount` / `missCount` / `totalZcCount` / `noiseRejectCount`, plus per-sector diagnostic arrays.
- `timing` — software-ZC period/flags; `sine`, `morph` — startup sub-state; `speedPi` — outer loop (parked).

`gspParams` (runtime parameters) and `gspDerived` (ISR-facing precomputed constants) feed the loop through a `RT_*` macro layer; the compile-time `#define` s are the seed defaults and define every formula.

2 Control State Machine & Startup

The motor is driven by an explicit state machine, `ESC_STATE_T` (`garuda_types.h:21`). Ordering is enforced by `_Static_assert`s because the voltage-fault gates compare the enum value with `>=` / `<=`.

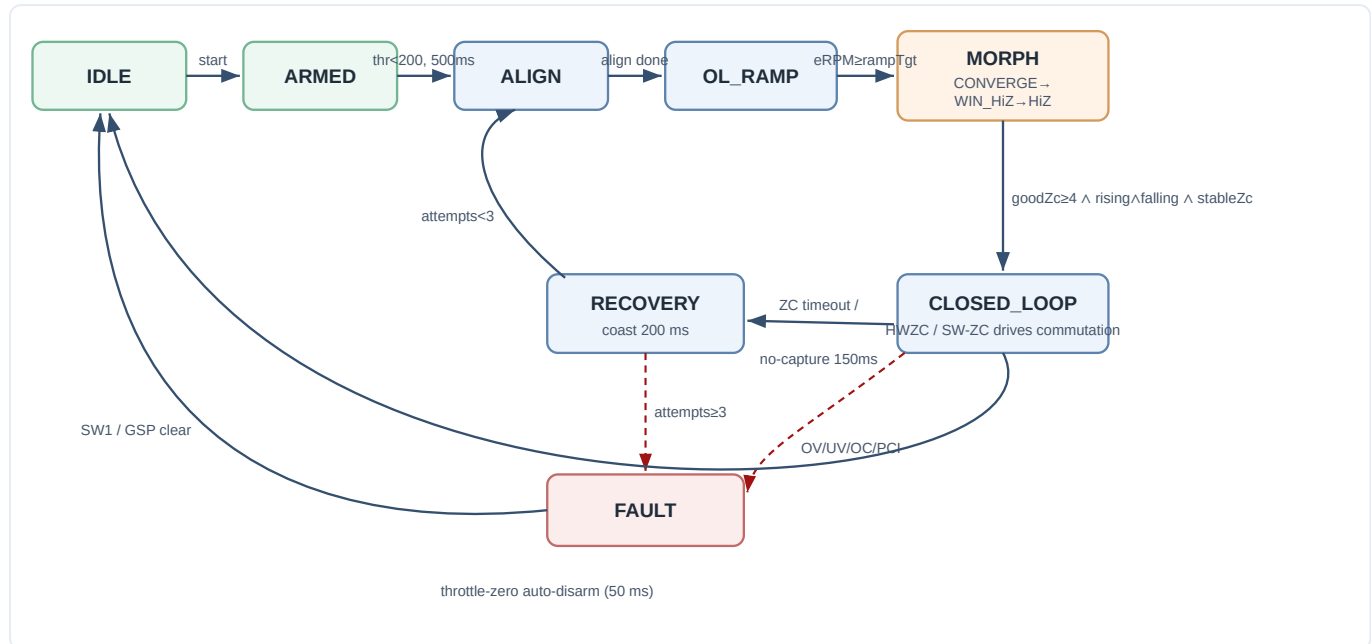


Figure 2. State graph (live SINE-startup + HWZC build). Green = idle/disarmed, blue = running, orange = handoff, red dashed = fault paths. AM32-kick, I-f and PLL-start are alternate strategies (§2.3) that bypass ALIGN/RAMP/MORPH.

2.1 States & transitions

State (value)	Bridge	Behaviour & exit condition
ESC_IDLE (0)	off	Disarmed. Resets OC auto-zero accumulators. → ARMED on SW1 / GSP start / RX auto-arm.
ESC_ARMED (1)	off	Throttle-zero gate; bus-ADC rest bias auto-zeroed for OC cal. → ALIGN when <code>throttle < 200</code> held 500 ms.
ESC_DETECT (2)	—	FOC-V2 auto-commissioning only; unreachable in 6-step.
ESC_ALIGN (3)	on	Rotor lock — holds a sine vector at Phase-A peak for <code>ALIGN_TIME_MS=500</code> . → OL_RAMP.
ESC_OL_RAMP (4)	on	Open-loop forced commutation; sine V/f field ramps <code>INITIAL_ERPM → rampTargetErpm</code> . → MORPH.
ESC_MORPH (5)	on	Sine → trapezoid blend + progressive Hi-Z + BEMF acquisition (sub-phases CONVERGE → WINDOWED_HIZ → HIZ). → CL on ZC lock, → FAULT on timeout.
ESC_CLOSED_LOOP (6)	on	Normal run. Commutation from detected ZC. Duty mapped from throttle with a <code>CL_IDLE_DUTY</code> floor.
ESC_RECOVERY (8)	off	Desync coast-down; counts <code>recoveryCounter</code> , then restarts or faults.
ESC_FAULT (9)	off	Latched; requires SW1 / GSP clear to return to IDLE.
ESC_IF_RAMP (10)	on	I-f current-controlled spin-up; compiled out in this build.

2.2 Align → open-loop ramp → morph → closed-loop

The bring-up is the firmware's most intricate sequence. In the active **sine-startup** build:

- ALIGN** — `STARTUP_SineInit()` seeds the sine angle to 90° (Phase-A peak), amplitude `RT_SINE_MIN_AMPLITUDE`, releases all PWM overrides (3-phase complementary), and holds for 500 ms while `STARTUP_SineAlign()` counts `alignCounter` down.
- OL_RAMP** — `STARTUP_SineRamp()` integrates an eRPM in Q16 (`erpmFrac += RT_SINE_ERPM_RAMP_RATE_Q16`), sets the angle increment, and linearly interpolates the amplitude from min to max — a constant-V/f forced field. Commutation here is *implicit*: the rotating sine field is the forced commutation, looked up from a 256-entry Q15 `sineTable256`.
- MORPH** — three sub-phases:
 - CONVERGE** blends $\text{duty} = \text{sine} \cdot (256 - \alpha) / 256 + \text{trap} \cdot \alpha / 256$ with α stepping 0 → 256 over `MORPH_CONVERGE_SECTORS=12` sectors. The trap floating phase uses a **virtual neutral** $(\text{trapDuty} + \text{MIN_DUTY}) / 2$, not 50 % — a 50 %-centred float against the ~5–7 % active rails would push ~5 V across 0.13 Ω = 39 A on low-resistance motors; the virtual neutral keeps the disconnection current ≈ 1 A.
 - WINDOWED_HIZ** opens a progressively wider Hi-Z window on the floating phase, polling BEMF inside it while still force-commutating, and adapts `timing.stepPeriod` toward the measured ZC interval ($\text{IIR} \gg 3$).

3. `HIZ` full Hi-Z; on `goodZcCount ≥ 4` with **both** rising and falling ZC working and a fresh ZC this tick, `zcSynced=true` and the state advances to `CLOSED_LOOP` — a bumpless "hot handoff" that skips ZC re-init and raises the CMP3 threshold to operational.

At the `CONVERGE → WINDOWED_HIZ` boundary the firmware deliberately drops `duty=MIN_DUTY` (coast) so near-zero voltage is applied while clean BEMF is acquired, and seeds the forced timing `handoff = TIMER1_T0_ADC_TICKS(rampStepPeriod)` (see §10 for the historical $\times 12/5$ bug in that macro).

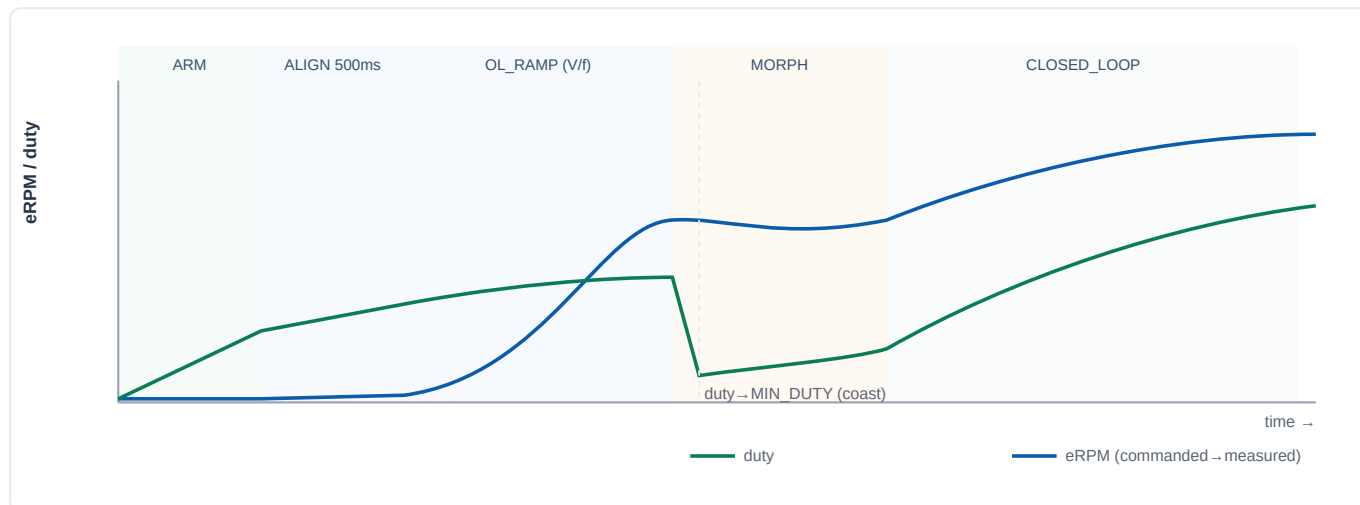


Figure 3. Startup envelope. eRPM is flat through ARM/ALIGN, ramps under V/f in OL_RAMP, dips slightly through MORPH, then follows throttle in CL. Duty rises under V/f, drops to `MIN_DUTY` at the morph coast notch, then slews up from the CL idle floor.

2.3 Startup strategies

Strategy	Flag	Mechanism	Active?
Sine V/f startup	<code>FEATURE_SINE_STARTUP=1</code>	ALIGN → RAMP → MORPH → CL as above	yes (profile 6)
Classic trap	<code>FEATURE_SINE_STARTUP=0</code>	step-0 hold → forced step ramp → CL	no
AM32 kick + listen	<code>FEATURE_AM32_STARTUP=1</code>	no align/ramp: ARMED → CL, one blind commutation, HWZC trusted from event 1	no (off for micros 6/7/8)
I-f current-controlled	<code>FEATURE_IF_STARTUP=1</code>	closed current loop on shunts + forced angle ramp	no (parked)
PLL-from-align	<code>FEATURE_PLL_STARTUP=1</code>	seed slow eRPM, blind accel, hand over on 6 consecutive plausible captures	no

2.4 Fault handling & recovery

Active detectors (all but PCI run in the ADC ISR):

- OV** — `vbusRaw > RT_VBUS_OVERVOLTAGE_ADC` ($3600 \approx 67\text{ V}$) for 3 samples → reset CMP3, disable outputs.
- UV** — `vbusRaw < uvThreshold` ($500 \approx 9.3\text{ V}$ run; relaxed to 400 pre-sync) for 3 samples.
- OC software** — `ibusRaw > OC_FAULT_ADC_VAL` debounced 3 consecutive ticks (a single rail-spiking sample used to latch a phantom).
- Board PCI** — hardware FPCI (combined OC+OV) in `_PWMInterrupt`; captures a frozen phase-current snapshot.
- Desync** — two watchdogs: software-ZC timeout, and an HWZC **no-capture watchdog** (`totalZcCount` stalls for 150 ms while the autonomous timer keeps commutating on a stale period).

On desync the bridge is disabled and the state → `RECOVERY` with `recoveryCounter = RT_DESYNC_COAST_COUNTS` (200 ms coast). After the rotor coasts, if attempts `< 3` the firmware re-runs `STARTUP_Init()` and re-enters `ALIGN`; a successful sync zeroes the attempt counter. `FAULT` is terminal until cleared.

3 PWM Generation & 6-Step Commutation

Three PWM generators drive the three half-bridges: **PG1 = Phase A (master)**, PG2 = B, PG3 = C (slaves). All are `MODESEL=4` center-aligned, complementary (`PMOD=0`) with hardware deadtime, and PG2/PG3 are phase-locked to PG1's start-of-cycle. The master holds `MPER = LOOPTIME_TCY` ; `MAX_DUTY = LOOPTIME_TCY · 99/100` .

3.1 Override-based 6-step drive

6-step does **not** modulate three phases. Each phase is in one of three states per sector, applied through the **override registers** (`PGxIOCON2`), written as single atomic 32-bit stores so inter-phase skew is ≈ 30 ns (vs ≈ 90 ns for read-modify-write bitfields — skew on a sector edge can spike $V_{bus} \cdot di/dt$ past the board fault threshold):

Encoding	PGxIOCON2	Effect
PWM_ACTIVE	0x00000000	Complementary PWM from <code>PGxDC</code> (HS modulated, LS complementary + deadtime)
LOW	0x00301000	H forced LOW, L forced HIGH → phase hard-tied to ground (return path)
FLOAT	0x00300000	Both H and L LOW → half-bridge high-Z (the BEMF-sensed floating phase)

`HAL_PWM_SetCommutationStep(step)` (`hal_pwm.c:1010`) indexes the commutation table and writes the three IOCON2 words.

`HAL_PWM_SetDutyCycle(duty)` writes the slaves first, master last (the master write broadcasts an UPDATE so all three load synchronously); only the `PWM_ACTIVE` phase's duty produces output.

Step	A	B	C	Float	ZC polarity
0	PWM	LOW	FLOAT	C	+1 rising
1	FLOAT	LOW	PWM	A	-1 falling
2	LOW	FLOAT	PWM	B	+1 rising
3	LOW	PWM	FLOAT	C	-1 falling
4	FLOAT	PWM	LOW	A	+1 rising
5	PWM	FLOAT	LOW	B	-1 falling

Even steps (0/2/4) are rising-ZC sectors; odd steps (1/3/5) are falling-ZC sectors. The floating phase rotates C → A → B → C → A → B and the polarity alternates.

`zcPolarity` (± 1 , `commutation.c:28`) is the master selector used everywhere.

3.2 ADC triggers from the PWM

Only PG1 emits ADC triggers. `PG1TRIGA` (`CAHALF=0`, `TRIGA=ADC_SAMPLING_POINT`) drives ADTR1, the primary BEMF/ V_{bus} / I_{bus} sample. On the MC510 `ADC_SAMPLING_POINT = LOOPTIME_TCY/2` — mid-period in the up-count half = the **freewheel / OFF-centre**, chosen by an empirical fire-time sweep (`MPER/2` reached 215 k eRPM vs 174 k + OC at pulse-centre). A second trigger `PG1TRIGB` (`CAHALF=1`, `TRIGB=0`) = period-centre, mid-ON) is used only by the bus-current experiments.

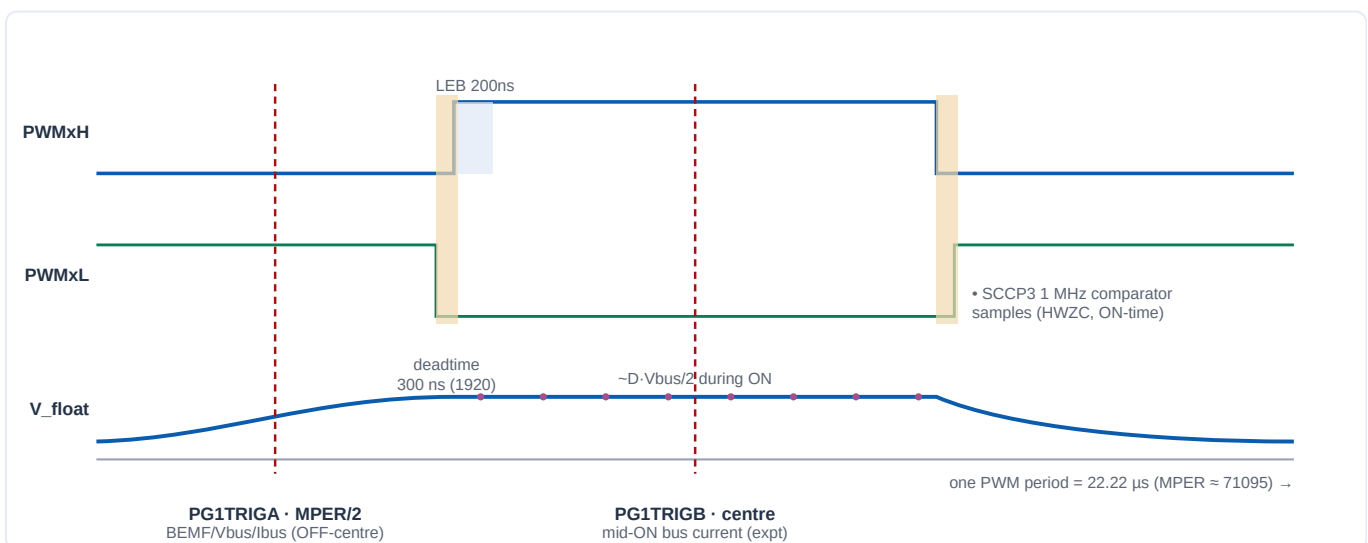


Figure 4. One center-aligned PWM period. The HS pulse sits in the cycle centre; the OFF-centre `PG1TRIGA` sample (red) lands in clean freewheel where the floating-phase BEMF is undistorted, while the HWZC comparator re-evaluates at 1 MHz during ON-time. Leading-edge blanking (LEB, 200 ns = 80 counts) masks the CMP3/FPC1 fault inputs across each switching edge.

4 Zero-Crossing Detection

This is the heart of the sensorless scheme. With one phase floating, its terminal voltage swings about the motor's virtual neutral and crosses it half-way between commutations. The firmware compares the floating-phase voltage against a computed neutral threshold; the crossing instant, stamped to 10 ns, is the timing reference the PLL locks to.

4.1 Two detection paths

Path	Source	Where it runs	Used when
Software ZC	ADC ISR samples <code>bemfRaw</code> once/PWM period and binary-thresholds it	45 kHz ADC ISR (<code>bemf_zc.c</code>)	low speed, before/around handoff
Hardware ZC	ADC on-chip digital comparator fires the instant the live 1 MHz conversion crosses <code>CMPLO</code>	priority-7 comparator + SCCP1 ISRs (<code>hwzc.c</code>)	above <code>hwzcCrossoverErpm</code>

The handoff is gated by `RT_HWZC_CROSSOVER_ERPM` (per profile: 1500 for the 2810/A2212, 24000 for the VEX "engages only where BEMF is solid") with `HWZC_HYSTERESIS_ERPM=500`. The switch is taken at a *true software-ZC deadline* so the hardware path inherits a clean rotor angle. `HWZC_Enable()` seeds `stepPeriodHR` from the software period and back-dates `lastZcStamp` by half a step so the first measured interval is $\approx$ one full step:

```
stepPeriodHR = HWZC_ADC_TO_SCCP2(timing.stepPeriod) | lastZcStamp = now - stepPeriodHR/2 hwzc.c:134,141
```

4.2 Neutral / threshold mathematics

During PWM-ON the floating terminal sits at roughly half the applied phase voltage, i.e. $V_{dc} \cdot D/2$. The threshold is recomputed every ISR tick from the bus ADC and the commanded duty:

```
rawThresh = (vbusRaw × duty) / ZC_DUTY_DIVISOR garuda_service.c:1040+
```

where the divisor selects the neutral model:

```
ZC_DUTY_DIVISOR = 2·L00PTIME_TCY → exact  $V_{dc} \cdot D/2$  (default, 2810 et al.)
ZC_DUTY_DIVISOR = (26·L00PTIME_TCY)/10 →  $V_{dc} \cdot D/2.6$  ( $\approx 23\%$  lower, micros 6/7/8) garuda_calc_params.h:189–192
```

Because `duty/L00PTIME_TCY = D` and `vbusRaw` is in ADC counts, `rawThresh` comes out in ADC counts. High-KV micros use the 2.6 divisor to pull the threshold down onto their tiny BEMF neutral. In closed loop the raw value is smoothed by a symmetric $\frac{1}{4}$ -gain IIR (re-seeded instantly on CL entry):

```
zcThreshSmooth += (rawThresh - zcThreshSmooth + 2) >> 2 ( $\alpha = \frac{1}{4}$ ,  $\tau \approx 0.17$  ms) garuda_service.c:1124
```

Optional refinements: `FEATURE_VIRTUAL_NEUTRAL` replaces the duty model with the measured average $(V_A + V_B + V_C)/3$;

`FEATURE_CL_DIFF_IDLE` lifts `thrDuty` by `3·MIN_DUTY` for differential idle; and `HWZC_THRESH_BIAS_DOWN` (=0 by default) can subtract a fixed bias for very-low-BEMF motors. A running peak `bemf.bemfDevPeak = max|float - neutral|` per sector is captured for the filter compensator (§6).

4.3 Sample point in the PWM cycle

Two distinct instants are sampled (Figure 4): the HWZC comparator channels convert **at 1 MHz during ON-time** (`TRG1SRC=34` = SCCP3), so the digital comparator continuously re-evaluates the ON-time crossing; the 45 kHz ISR `bemfRaw` is the **OFF-centre freewheel** sample (`TRG1SRC=4` = PG1TRIGA), used by software ZC, the falling-SW detector, and `bemfDevPeak`. After commutation, detection is masked through a **blanking** window to skip diode-freewheel/demag ringing:

```
HW: blankTicks = period × HWZC_BLANKING_PERCENT/100 (14 %), min 1 μs
SW: blankBase = stepPeriod × ZC_BLANKING_PERCENT/100 (3 %); + demag extra when D > zcDemagDutyThresh, capped at
stepPeriod/4 hwzc.c:286 · bemf_zc.c
```

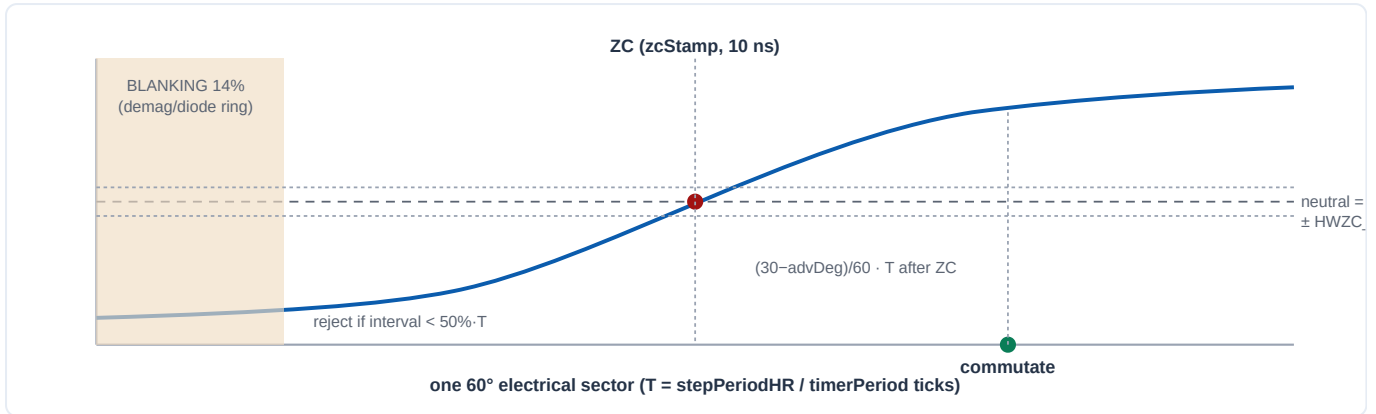



Figure 5. A single rising sector. BEMF ramps across the neutral threshold at mid-sector. The first 14 % is blanked, the threshold carries a $\pm$ deadband, the crossing is the ZC, and commutation fires $(30 - \text{advDeg})/60$ of the sector later.

4.4 Falling-SW per-polarity detection

During PWM-ON the driven phase capacitively couples into the floating phase and biases it *up*. This **assists rising-ZC** detection but **swamps the falling crossing**, so on odd (falling) sectors the ON-time comparator goes silent at speed (100 % of misses were on odd sectors). Clean falling BEMF only exists in the PWM-OFF freewheel — exactly the `bemfRaw` OFF-centre sample.

With `FEATURE_HWZC_FALLING_SW=1` the firmware arms the HW comparator **only on rising sectors** (`hwzc.c:341–350`); falling sectors are detected by a software compare on the OFF-centre sample:

```
// falling-SW: in WATCHING, on a falling sector, OFF-centre sample vfo
fth = HWZC_ApplyFilterComp(thresh, falling) - deadband;
if (vfo < fth && intoSector > timerPeriod>>2 && !captureValid) {
    lastCaptureHR = zc; captureValid = true; // one accept per sector
    goodZcCount++; totalZcCount++;
}
```

A speed gate `HWZC_FALLING_SW_MAX_ERPM` (per profile: 35 k A2212, 45 k VEX, 50 k 1407, 70 k 2810) disengages the falling-SW path above it — at high eRPM the OFF-centre sample (~ 1 PWM period resolution) is too coarse and the rising comparator can carry both polarities. A mirror `FEATURE_HWZC_LOWSPD_OFFCTR` detects *both* polarities on the OFF-centre sample at low speed, removing the ON-time min-duty floor and enabling a low-duty CL handoff.

4.5 Validation & rejection logic

`HWZC_OnZcDetected()` (`hwzc.c:402`) runs cheap gates first:

1. **First-ZC bypass** — the very first ZC after enable skips the plausibility gate and IIR (the seeded `lastZcStamp` is a half-step guess; the first interval can be 10–190 % of a step). It just anchors the stamp.
2. **PWM-state gate** (`FEATURE_HWZC_PWM_GATE`) — reads the H-side GPIO of the PWMing phase; if PWM is OFF, reject. ON-time crossings ($\approx D \cdot V_{\text{bus}}/2$) and OFF-time pseudo-crossings (≈ 0 V) are different regimes; mixing them gives 60° slip $\rightarrow$ 22 A spike.
3. **Interval / plausibility gate** — reject if the interval is implausibly short:

```
interval < max( stepPeriodHR × HWZC_MIN_INTERVAL_PCT/100 , RT_HWZC_MIN_STEP_TICKS ) → reject
```

`hwzc.c:501`

`HWZC_MIN_INTERVAL_PCT = 50`. A shorter interval implies the motor > doubled speed in one sector — a PWM-ripple re-fire.

4. **Deadband / hysteresis** — the comparator `CMPL0` is shifted toward the signal by `HWZC_CMP_DEADBAND` (2 counts for micros, else 4): rising `thresh+DB`, falling `thresh-DB`, so ripple straddling neutral doesn't chatter.
5. **Coherence verify** (`FEATURE_HWZC_VERIFY_READS`) — wait one ADC conversion ($\sim 1 \mu\text{s}$) then re-read a fresh sample. A real crossing stays on the crossed side; a ripple phantom flips back $\rightarrow$ reject. (Re-reading immediately would only see the trigger sample, hence wait-first.)

What "reject %" (`rej`) means. Rejections fire mostly during PWM-OFF, so $\text{rej} \approx (1 - \text{duty})$ — the OFF-window fraction. A high `rej` is *not* guessing: it is the comparator correctly firing on filtered freewheel ripple and being rejected. The meaningful signal is accept-vs-miss per sector (`dbgPiMissBySector[6]` proved the falling-polarity masking).

On accept the IIR updates only when the ZC is consecutive (`stepsSinceLastHwZc==1`) and `goodZcCount ≥ HWZC_IIR_FREEZE_ZC_COUNT` (3 — the freeze prevents a mistimed morph-exit ZC from ratcheting the period to floor).

5 Commutation Timing: PLL, Sector-PI & IIR

The commutation period is tracked in 10 ns SCCP2 ticks. Conversion to/from electrical RPM folds the 100 MHz clock, 6 sectors/erev and 60 s/min into a single constant:

$$\text{HWZC TICKS TO ERPM}(t) = 1 \times 10^9 / t \quad | \quad \text{HWZC ERPM TO TICKS}(e) = 1 \times 10^9 / e$$

garuda_calc_params.h:417

So a `stepPeriodHR` of 5000 ticks (50 μ s) = 200 000 eRPM. The build runs `FEATURE_HWZC_SECTOR_PI=1`, so the **autonomous-timer + PI** architecture is live; the reactive IIR path runs only when PI is compiled off, and as the seed.

5.1 The autonomous-timer PLL

In sector-PI mode, detection is decoupled from commutation. SSCP1 (a 32-bit one-shot) is armed in `HWZC_0nBlankingExpired` to fire at `lastCommStamp + timerPeriod`; that expiry dispatches to `HWZC_0nPiPeriodExpired`, which runs the PI correction and then commutates. A ZC arriving between merely timestamps `lastCaptureHR` — it does not reschedule. A missed ZC simply leaves `timerPeriod` unchanged (robust by design). This structure lifted the ceiling from ~204 k (reactive wobble) to 240 k+.

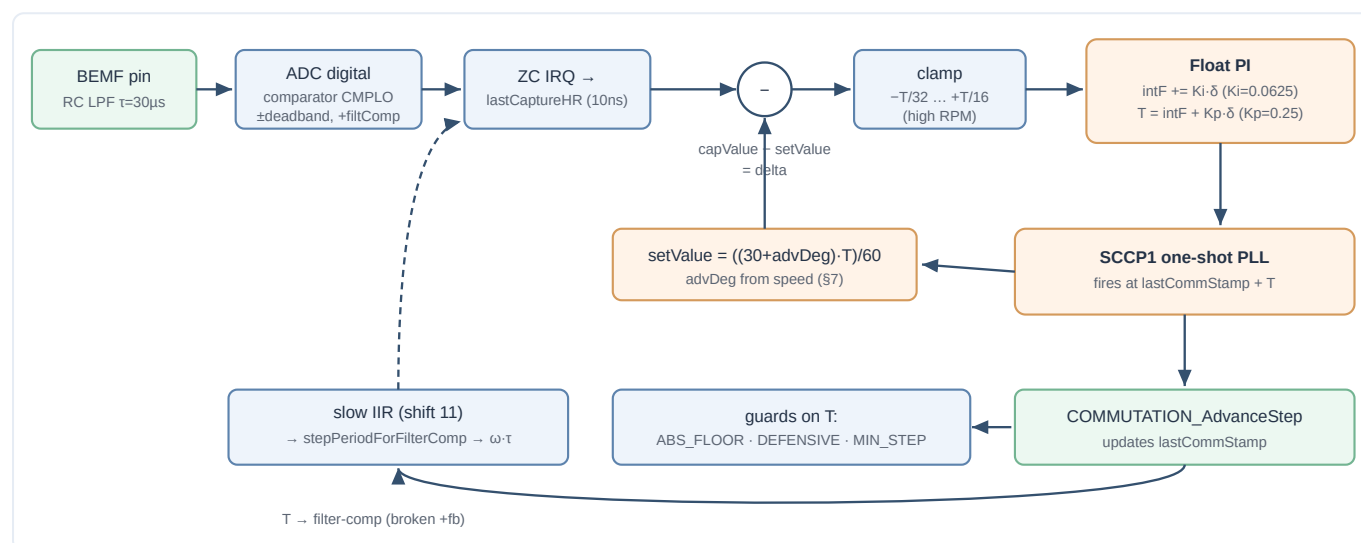


Figure 6. The sector-PI PLL. The ZC capture is differenced against the advance-derived setpoint; the clamped error drives a float PI that sets the commutation period T ; an autonomous one-shot fires commutation at `lastCommStamp+T`. A deliberately slow IIR breaks the filter-comp positive-feedback path.

5.2 The sector PI controller

The controlled quantity is the position of the captured ZC within the sector. With the autonomous timer firing `OnPiPeriodExpired` just before commutation, the elapsed time from the sector's start commutation to the ZC is

```
capValue = lastCaptureHR - lastCommStamp    (modular u32)
```

hwzc-c:828

The setpoint is the expected ZC position, advanced by the schedule (O8 fixed-point):

$$\text{advFp8} = ((30 + \text{advDeg}) \cdot 256) / 60 \in [128, 256] \quad | \quad \text{setValue} = (\text{advFp8} \cdot T) \gg 8 \approx ((30 + \text{advDeg}) / 60) \cdot T$$

hwzc.c:804.829

$\text{delta} = \text{capValue} - \text{setValue}$ (signed phase error; $\rightarrow 0$ at lock)

hwzc.c:849

The error is clamped **asymmetrically and RPM-dependently** before the loop:

T < HWZC_PI_CLAMP_HIGH_TICKS (≥ 150 k eRPM): +cap = T/16 , -cap = T/32
 else (low/mid RPM): symmetric $\pm T/8$

hwzc.c:851-859

A **negative** delta means "shrink the period" (commutate faster). At the BEMF ceiling the rotor cannot accelerate, so a negative drift would commutate ahead of the rotor → wrong-angle → 22 A spike → UV cascade. The tighter negative clamp (T/32) blocks that while the looser positive clamp (T/16) keeps the loop responsive to genuine slowdowns. A separate `FEATURE_HWZC_HANDOFF_DAMP` hard-limits negative delta to `T>6` for the first 24 commutations after CL entry.

The active loop is a parallel-form float PI, gains specified in PLL form ($\omega_n=0.25$, $\zeta=0.5$):

```
integratorF += Ki.delta ,    Ki = ωn2 = 0.0625
T = integratorF + Kp.delta ,    Kp = 2ζωn = 0.25 ,    both clamped ≥ RT_HWZC_MIN_STEP_TICKS
```

hwzc.c:890-898

Because ω_n is in rad *per sector-event*, the loop's absolute bandwidth scales with rotor speed (≈ 56 Hz at 14 k eRPM, ≈ 915 Hz at 230 k) — it widens with speed, matching the rotor timescale. The **defensive** sub-mode (`FEATURE_HWZC_PI_DEFENSIVE`) tracks silence: after 6 silent sectors it walks the integrator *larger* by 1 %/event so a slipping rotor can recapture, exiting after 2 good captures — a soft self-heal that replaced a hard desync watchdog.

5.3 The IIR period filter (reactive seed)

In the reactive path, the period is a first-order IIR over measured intervals:

```
newPeriod = (3·stepPeriodHR + interval) / 4    ( $\alpha = \frac{1}{4}$ ) , floored at RT_HWZC_MIN_STEP_TICKS
```

hwzc.c:608

guarded by: **(1)** consecutive-only (`stepsSinceLastHwZc==1` , so multi-step timeout intervals don't contaminate it); **(2)** freeze for the first 3 good ZCs after enable; **(3)** a seqlock (`writeSeq++ ... writeSeq++`) so a lower-priority reader never tears the 32-bit write. In PI mode this IIR is paused — `stepPeriodHR` is mirrored from the PI's `timerPeriod` , and a *separate* slow IIR (shift 11, ≈ 11 ms) feeds the filter compensator.

6 RC Filter-Lag Compensation — Full Mathematics

The board's BEMF sense pin sits behind a passive RC low-pass ($\tau \approx 30 \mu\text{s}$). A first-order low-pass delays the apparent zero-crossing by a phase angle that grows with rotor speed, so the comparator fires *late*. The compensator shifts the comparator threshold *away from neutral* by exactly that lag so the comparator trips early, recovering the true pre-filter ZC. This section gives the complete math, including the ZC threshold-bias term.

6.1 The physical lag

A first-order low-pass with time constant τ introduces, at electrical frequency ω_e , a phase lag $\phi = \arctan(\omega_e \tau) \approx \omega_e \tau$ for small angles. The firmware works in the small-angle regime and caps the term (below). ω_e follows directly from the sector period: with $\text{stepPeriodHR} = 1e9/\text{eRPM}$ (HR ticks) and $f_{\text{elec}} = \text{eRPM}/60$, the product $\omega_e \tau$ is computed as a Q15 fixed-point quantity.

6.2 The threshold-bias formula

The compensator `HWZC_ApplyFilterComp(thresh, risingZc)` (`hwzc.h:48`) computes the bias step by step:

```
(1) omegaTauQ15 = HWZC_FILTER_K_Q15 / stepP
(2) if omegaTauQ15 > HWZC_FILTER_MAX_OMEGA_Q15 (24000) → clamp to 24000
(3) amp = bemf.zcAmpForFilterComp ; if amp==0 → amp = thresh
(4) ampPct = risingZc ? HWZC_FILTER_AMP_PCT : HWZC_FILTER_AMP_PCT_FALLING (both 50)
(5) offset = ((amp × omegaTauQ15) >> 15) × ampPct / 100
(6) if offset > HWZC_FILTER_MAX_OFFSET (600) → clamp to 600
(7) rising: return thresh - offset | falling: return thresh + offset
```

`hwzc.h:70-98`

The constant `HWZC_FILTER_K_Q15` folds τ , the SCCP clock and the Q15 scale so that step (1) yields $\omega_e \tau$ in Q15 directly:

$$\text{HWZC_FILTER_K_Q15} = 2\pi \cdot 10^9 \cdot \tau \cdot 32768 / 60 = 2\pi \cdot 10^9 \cdot 30 \times 10^{-6} \cdot 32768 / 60 \approx \mathbf{102\ 943\ 706}$$

`garuda_config.h:1236`

Why this works: stepP (HR ticks) = $10^9/\text{eRPM}$ and $f_{\text{elec}} = \text{eRPM}/60$, so

$$\omega_e \tau = K/\text{stepP} = [2\pi \cdot 10^9 \cdot \tau \cdot 32768 / 60] \cdot [\text{eRPM} / 10^9] = 2\pi \cdot (\text{eRPM} / 60) \cdot \tau \cdot 32768 = (\omega_e \tau) \cdot 32768$$

i.e. the division by stepP turns the folded constant into $\omega_e \tau$ expressed in Q15 ($32768 = 1.0 \text{ rad}$). The cap of 24000 limits $\omega_e \tau$ to $\approx 0.73 \text{ rad}$ ($\approx 42^\circ$), preventing the offset blowing up at very high RPM (the cap engages around 200 k).

The amplitude scaling (step 5) converts the phase offset into an ADC-count threshold shift. For a near-sinusoidal BEMF of peak amplitude amp crossing zero, a small time/phase shift $\delta\phi$ moves the crossing voltage by $\approx \text{amp} \cdot \delta\phi$; here $\delta\phi = \omega_e \tau$ (Q15), so $(\text{amp} \times \omega_e \tau) \gg 15 = \text{amp} \cdot \omega_e \tau$ is the count-shift needed to trip the comparator exactly one filter-lag early. The $\times \text{ampPct}/100$ (50 %) is an empirical de-rating — the linear small-angle model over-predicts, and 50 % matched the bench at the proven speeds.

6.3 The ZC threshold-bias chain end-to-end

It is worth following the full threshold a comparator actually sees, because three distinct biases stack:

$$\text{CMPLO}_{\text{final}} = [\text{zcThreshold}] \mp \text{offset}_{\text{filtComp}} \pm \text{HWZC_CMP_DEADBAND}$$

- **zcThreshold** — the duty-model neutral $\text{vbusRaw} \cdot \text{duty} / \text{ZC_DUTY_DIVISOR}$, $\frac{1}{4}$ -IIR-smoothed (§4.2).
- $\mp \text{offset}_{\text{filtComp}}$ — pulls the threshold *toward the oncoming signal* (down for rising, up for falling) by the filter-lag amount above, so the comparator fires early enough to cancel the RC delay.
- $\pm \text{deadband}$ — pushes the threshold a fixed 2–4 counts *away* from neutral in the crossing direction (rising $+\text{DB}$, falling $-\text{DB}$) so PWM ripple straddling neutral cannot chatter the comparator.

Sign convention together: for a **rising** sector the net is $\text{thresh} - \text{offset} + \text{DB}$; for a **falling** sector $\text{thresh} + \text{offset} - \text{DB}$. The two biases act in opposite senses by design — the filter-comp advances the trip, the deadband guards against chatter — and the same `HWZC_ApplyFilterComp` result is used at every CMPLO write site (initial arm in `HWZC_OnCommutation`, the live ISR refresh, the verify path, and the falling-SW software compare) so all paths stay consistent.

6.4 The amplitude source — why measured, not duty-derived

The offset uses `zcAmpForFilterComp` — the *measured* BEMF amplitude (a slow $\approx 46 \text{ ms}$ IIR of `bemfDevPeak`) — **not** the live `zcThreshold`. This is critical: $\text{zcThreshold} \propto \text{duty}$, so feeding it into the offset would make the compensation duty-dependent and create a regen positive-feedback path (more duty $\rightarrow$ bigger offset $\rightarrow$ earlier fire $\rightarrow$ period shrinks $\rightarrow$ more duty). The measured amplitude lags with rotor inertia and reflects the *real* BEMF magnitude the RC lag acts on. This is the "`bemfDevPeak` vs `zcThreshold`" fix that centred the ZC from $\sim 15\%$ to $\sim 50\%$ of the sector.

A second positive-feedback break is in the *period* source: in PI mode `stepP` for step (1) is the slow IIR `stepPeriodForFilterComp` (shift 11), not the live `timerPeriod`. Otherwise a PI period-shrink would grow $\omega_e \tau \rightarrow$ grow the offset $\rightarrow$ fire earlier $\rightarrow$ shrink the period further. The slow

IIR keeps that loop gain $\ll 1$.

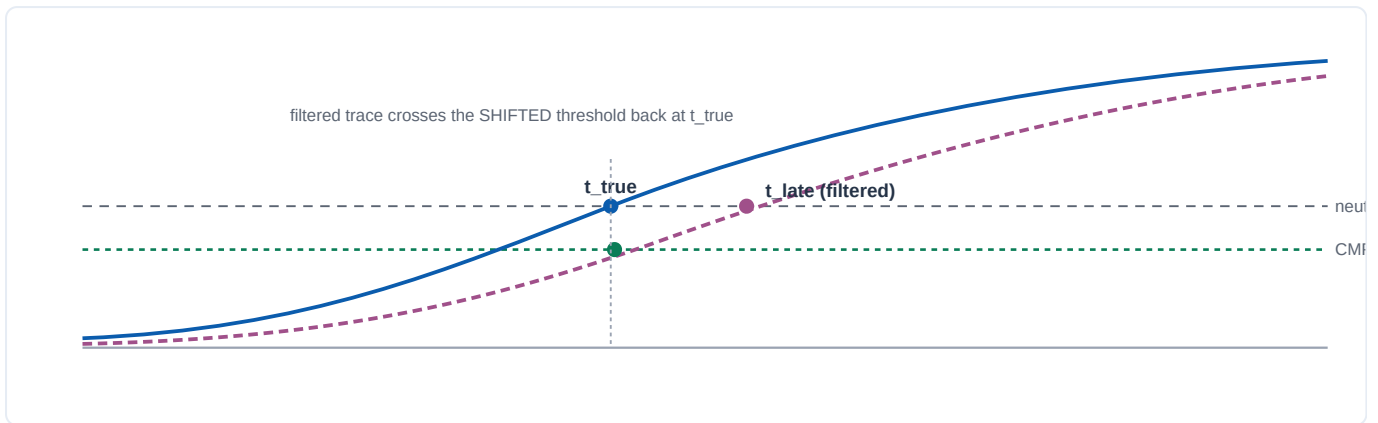


Figure 7. Filter-lag compensation (rising sector). The RC-filtered BEMF (dashed magenta) crosses neutral late at t_{late} ; lowering the comparator threshold by $\text{offset} = \text{amp} \cdot \omega_e \tau \cdot 50\%$ makes the filtered trace reach the shifted threshold (green) back at t_{true} . Falling sectors raise the threshold by the mirror amount.

7 Timing Advance

This is 6-step commutation phase advance, NOT field weakening. There is no d-axis current and no space-vector modulation. Advance simply moves the commutation instant earlier in the sector to compensate the inductive lag (phase current cannot switch instantly) plus the detection/filter latency, which lets the motor run past its nominal no-load speed.

The advance angle is a linear interpolation of speed, identical in all three sites (`ComputeCommDelay`, `HWZC_OnPiPeriodExpired`, the SW path):

```
eRPM ≤ RT_RAMP_TARGET_ERPM → advDeg = TIMING_ADVANCE_MIN_DEG (0°)
eRPM ≥ RT_TIMING_ADV_FULL_ERPM → advDeg = RT_TIMING_ADV_MAX_DEG
else → advDeg = MIN + (MAX-MIN) · (eRPM - RAMP_TARGET) / (FULL - RAMP_TARGET) hwzc.c:307-323
```

Advance is baked into **both** sides of the loop: in reactive mode the schedule fires early — `commDelay = stepPeriodHR · (30 - advDeg) / 60` — and in PI mode the setpoint shifts — `setValue = ((30 + advDeg) · T) / 60` — so the loop holds the same phase relationship. The endpoint `RT_TIMING_ADV_FULL_ERPM` is **decoupled** from the speed cap (it defaults to `RT_MAX_CLOSED_LOOP_ERPM` but can be overridden), so a motor operating far below its cap still gets meaningful advance growth instead of a near-flat ramp.

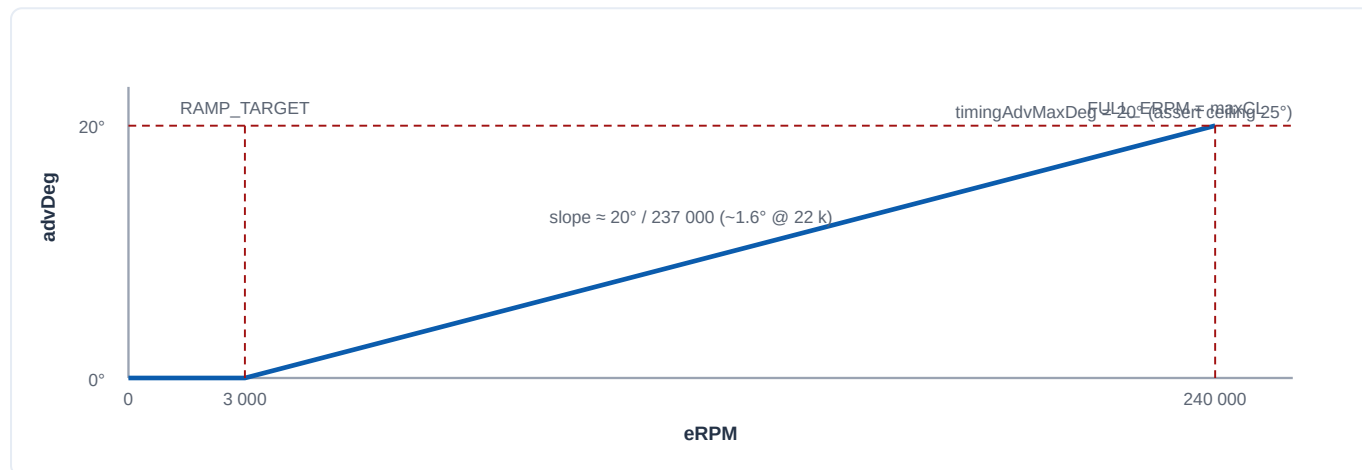


Figure 8. Advance ramp for the reference profile: 0° up to `RAMP_TARGET_ERPM` (3000), linear to `timingAdvMaxDeg` (20°) at `RT_TIMING_ADV_FULL_ERPM` (240 000), flat above. The same eRPM (240 000) also sets the period floor `RT_HWZC_MIN_STEP_TICKS = 1e9/240000 ≈ 4167 ticks`.

8 ABS_FLOOR False-Lock Guard & the λ Math

The relative gates of §4.5 are ratiometric; they cannot catch a cumulative harmonic ratchet where the PI slowly walks onto a half- or third-period phantom lock. `FEATURE_HWZC_ABS_FLOOR` adds an **absolute** operating-point floor from no-load physics: the rotor cannot spin faster than the no-load speed for the present duty and bus voltage.

The no-load sector period in HR ticks is computed from the motor's flux constant λ (= `focKeUvSRad`), in $\mu\text{V}\cdot\text{s}/\text{rad}$:

```
dutyFrac = duty / LOOPTIME_TCY ; vbus_v = vbusRaw × VBUS_SCALE_V_PER_COUNT ; λ = gspParams.focKeUvSRad
Pff = 181.380 × λ / (vbus_v × dutyFrac) [HR ticks]
floorP = Pff × (100 / HWZC_ABS_FLOOR_OVERSPEED_PCT) (overspeed 130 → allow 130 % of no-load)
```

hwzc.c:920–953

The constant **181.380** folds the SCCP clock (10^8), the 6 sectors/rev, the 2π , and the BEMF $\leftrightarrow$ period algebra so `Pff` comes out directly in 10 ns ticks. The guard is applied only above `HWZC_ABS_FLOOR_MIN_DUTYFRAC=0.03` and `vbus_v > 6`.

SHRINK-ONLY: clamp only when (timerPeriod < floorP) AND (timerPeriod < T) – the PI is *driving* the period below the floor.

hwzc.c:942

A period already below the floor but *growing* is a genuine high-speed coast-down through the floor and is left alone (the rotor really is that fast); only an accelerating shrink at idle duty is provably a false-lock.

Why a wrong λ pins the motor. `Pff ∝ λ`. If λ is too high (e.g. the 2810's 583 used on a motor whose true flux is 230), `Pff` and `floorP` inflate, the floor sits above the real no-load period, and the guard clamps the period up to `floorP` every sector — pinning eRPM at $130\% \cdot (\lambda_{\text{true}}/\lambda_{\text{set}}) = 130\% \cdot (230/583) \approx 51\%$ of no-load with large reactive current. This is exactly the VEX "half-speed/high-current" symptom; the fix was setting `focKeUvSRad = 5.5132 × 106 / (KV · PP)`. The same formula seeds the PI integrator at handoff (one-shot, slow-only).

9 Protection & Auxiliary Control

9.1 The overcurrent ladder

Bus current is sensed on the OA3 op-amp off the low-side DC-link shunt. The count $\leftrightarrow$ amp frame is defined once:

```
OC_TRIP_MV(ma) = OC_VREF_MV + (ma * OC_SHUNT_MOHM * OC_GAIN_X100 / 100000) = 1650 + ma * 3.2495 / 100000
OC_MV_TO_COUNTS(mv) = mv * 4096 / 3300 → ≈ 93 counts/A, 2048 = 0 A
```

garuda_calc_params.h:480

Four layers act in sequence (reference profile values):

Layer	mA	counts	Mechanism
HW CMP3 chop (run)	9 000	2883	Analog cycle-by-cycle CLPCI truncation of each PWM pulse
HW CMP3 chop (startup)	10 000	2976	Elevated during align/OL/morph so torque isn't chopped
SW soft limit	13 000	3255	Proportional duty reduction (a slow backstop, sits above the chop here)
SW hard fault	14 000	3347	3-consecutive-sample debounce → <code>FAULT_OVERCURRENT</code>

The SW soft limiter reduces duty in proportion to the excess: $\text{reduction} = \text{excess} \cdot (\text{duty} - \text{MIN_DUTY}) / \text{range}$, guarded by $\text{duty} > \text{MIN_DUTY}$ to prevent unsigned underflow.

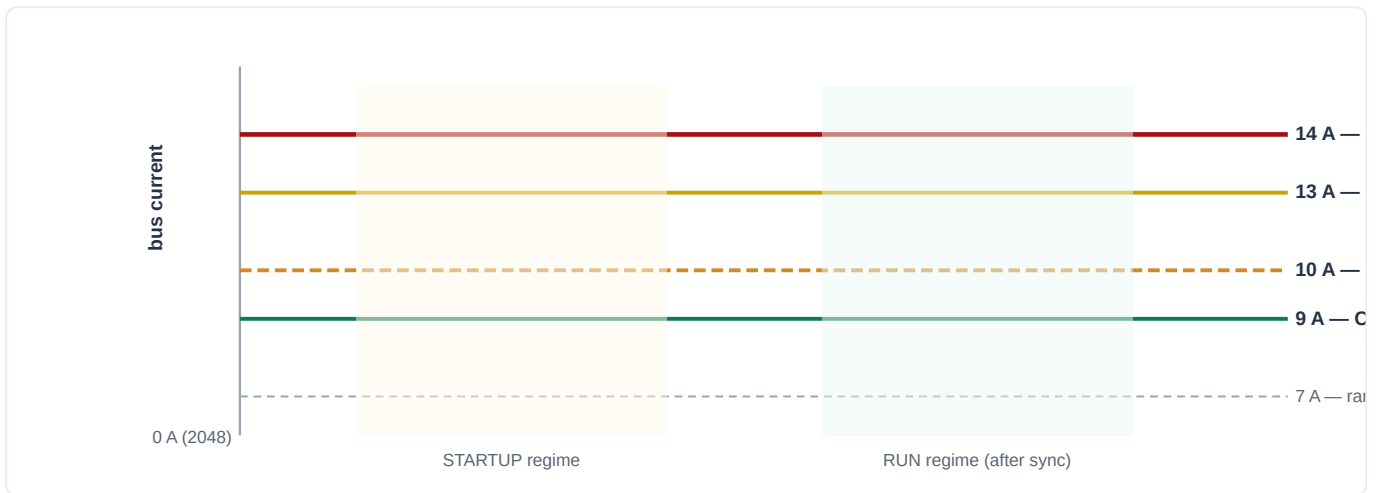


Figure 9. The four-layer OC ladder. The HW chop is the fast cycle-by-cycle limiter (10 A during startup, 9 A in run); the SW soft/hard layers are slower backstops above it. On a low-resistance micro the soft limit is deliberately placed above the chop.

9.2 OC auto-zero bias

The OC math assumes a 1.65 V / 2048-count rest, but on this board the OA3 chain rests at ≈ 78 counts. `FEATURE_OC_AUTOZERO` measures the true rest during ARMED (bridge off, 64 samples) and re-centres everything into the 2048 frame — but **only if** the sample spread ≤ 16 counts (a quiescence gate; a quick re-arm with the rotor still coasting puts regen ripple on the bus and would latch a bias ≈ 7 A wrong). Three things are corrected with the same delta $(2048 - g_ocBiasAdc)$:

```
ibusRaw_corrected = rawBus + (2048 - g_ocBiasAdc) (freewheel/non-inverted path)
CMP3 DAC: v = dacVal - (2048 - g_ocBiasAdc) (single choke HAL_CMP3_SetThreshold)
```

garuda_service.c:1596 · hal_comparator.c:271

Until the cal runs, `g_ocBiasAdc = 2048` makes the shift a no-op — a graceful legacy fallback.

9.3 Bus-voltage tiers

Scale: $\text{VBUS_SCALE_V_PER_COUNT} = 3.3 \cdot 23.2 / 4095 = 0.018696 \text{ V/count}$. All tiers use a 3-sample filter. Two sticky-hysteresis FSMs sit between the regen brake and the hard OV fault:

Tier	ADC / volts	Action
Hard OV fault	3600 / ~ 67 V	3-sample → <code>FAULT</code> , reset CMP3, outputs off
Emergency hold (on/off)	1620 / 1450 (~ 30 / ~ 27 V)	freeze slew-down entirely (3 V hyst, min 20 ms)
Regen brake (on/off)	1500 / 1290 (~ 28 / ~ 24 V)	slew-down $\div 16$ (4 V hyst, min 10 ms)
Vbus sag (limit/recover)	900 / 1000 (~ 17 / ~ 19 V)	reduce duty (depth-8) $\gg 4$
UV fault (run / startup)	500 / 400 (~ 9.3 / ~ 7.4 V)	3-sample → <code>FAULT</code> (relaxed pre-sync)

The regen brake gets first chance (slow the deceleration so the bus cap absorbs the energy); if Vbus still climbs, the emergency hold freezes slew-down completely, breaking the positive-feedback loop that previously drove a 47.8 V PSU-OV trip. Only the regen direction is frozen — rising duty (which consumes regen) still passes.

9.4 Outer speed PI (parked)

`FEATURE_SPEED_PI=0`. A per-ZC eRPM control loop exists (`speed_pi.c`) working in eRPM space (constant plant gain ≈ 3.2 eRPM/tick vs $16\times$ variation in period space). Algorithm per ZC: throttle $\rightarrow$ target map, measured = $1e9/\text{stepPeriodHR}$, a 5 % deadband (error *shifted* not clamped), a bench-fit feedforward `ff = 0.310 * target + 400`, a PI on the residual (`Kp=0.03, Ki=0.001`) with a tight ± 8000 -tick integrator clamp and back-calculation anti-windup. It is parked because the inner HWZC timing PI and this outer loop form a nested loop that cascades to PSU-OV $\rightarrow$ phantom 200 k $\rightarrow$ UV above ~ 80 k; revival needs an absolute output slew limiter first. Its telemetry (`speedPiTarget/LastError/OutputDuty/IntegratorF`) is still plumbed.

10 GSP Host Protocol & Parameter System

The Garuda Serial Protocol is a binary command/telemetry API over UART1 at 115.2 kbps, polled (no UART ISR), pumped from `main()` via `GSP_Service()` (PumpRx → ParserProcess → TelemTick → PumpTx). It is the host-facing API for the GUI and MCP tooling.

10.1 Wire protocol & CRC

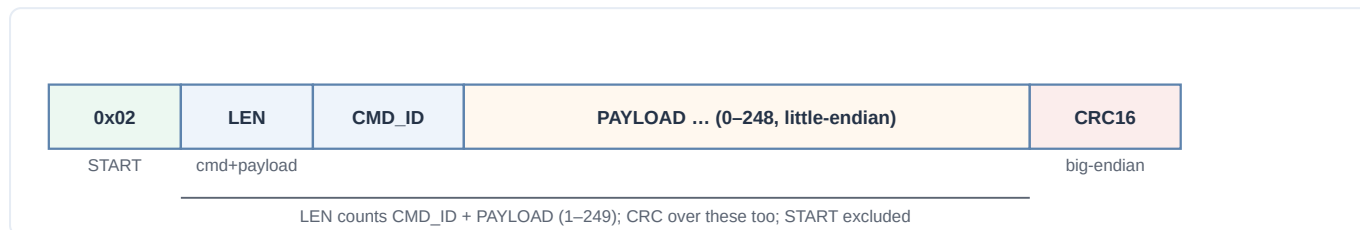


Figure 10. GSP frame. `LEN` counts `CMD_ID` + payload (not `START` or `CRC`). The `CRC` is CRC-16-CCITT (poly `0x1021`, init `0xFFFF`, MSB-first, no final XOR/reflection) over `LEN` + `CMD_ID` + payload, sent high byte first.

Versioning is pull-based: the host issues `GET_INFO` (`0x01`) and reads `protocolVersion` (`GSP_PROT0COL_VERSION=3`) from the 24-byte INFO frame, then decodes everything else accordingly. Snapshot fields are append-only so older decoders parse the prefix and ignore the tail. On CRC mismatch the frame is silently dropped; the host retries.

10.2 Command set

Opcode	Name	Req payload	Response
0x00	PING	0	echo
0x01	GET_INFO	0	INFO (24 B)
0x02	GET_SNAPSHOT	0	SNAPSHOT (248 B)
0x03 / 0x04 / 0x05	START / STOP / CLEAR_FAULT	0	ACK / ERR_WRONG_STATE
0x06	SET_THROTTLE	2 (u16 0–2000)	ACK
0x07	SET_THROTTLE_SRC	1 (u8)	ACK
0x08	HEARTBEAT	0	ACK
0x10	GET_PARAM	2 (u16 id)	id+value (6 B)
0x11	SET_PARAM	6 (id u16 + value u32)	id+value (6 B) / ERR
0x12 / 0x13	SAVE_CONFIG / LOAD_DEFAULTS	0	ACK / ERR_EEPROM_THROTTLED
0x14 / 0x15	TELEM_START / STOP	1 (Hz) / 0	actualRate / ACK
0x16	GET_PARAM_LIST	0/1 (startIdx)	paginated descriptors (≤20/page)
0x17	LOAD_PROFILE	1 (u8)	profileId / ERR
0x20	AUTO_DETECT	0	ACK / ERR_WRONG_STATE
0x26 / 0x27	GET_RX_STATUS / RX_INJECT	0 / 4	status / ACK
0x30–0x32	SCOPE_ARM / STATUS / READ	8 / 0 / 2	burst-scope
0x80	TELEM_FRAME	—	seq(u16)+SNAPSHOT (unsolicited stream)
0xFF	ERROR	—	errCode(u8)[+aux]

Errors: 0x01 UNKNOWN_CMD, 0x02 BAD_LENGTH, 0x03 BUSY, 0x04 WRONG_STATE, 0x05 OUT_OF_RANGE, 0x06 UNKNOWN_PARAM, 0x07 CROSS_VALIDATION, 0x08 EEPROM_THROTTLED. Motor-control commands set intent flags (`gspStartIntent` etc.) consumed by the state machine, decoupling the polled host loop from the control ISR.

10.3 Parameter & descriptor system

The 58-entry descriptor table (`paramDescriptors[]`) carries `{id, type, group, min, max, offset, size}`; every `SET_PARAM` is bounds-checked then **cross-validated** for bilateral invariants (`initialErpm < rampTargetErpm < maxClosedLoopErpm`; `ocSwLimit < ocLimit ≤ ocFault`; `vbusUv < vbus0v`; ...), then `GSP_RecomputeDerived()` refreshes the ISR-facing constants. Selected control parameters:

ID	Name	Type	Min	Max	Units / role
0x15	RAMP_TARGET_ERPM	u16	500	20000	OL ramp endpoint
0x22	TIMING_ADV_MAX_DEG	u8	0	45	max commutation advance °
0x30	HWZC_CROSSOVER_ERPM	u16	500	20000	eRPM to enable HWZC

0x53	MAX_CL_ERPM	u32	5000	350000	PI period-floor clamp + advance anchor
0x20	CL_IDLE_DUTY_PCT	u8	0	30	CL idle duty floor %
0x58 / 0x59	OC_LIMIT / OC_STARTUP_MA	u16	501 / 5000	22000	CMP3 chop (run / startup), live-tunable
0x41 / 0x42	OC_FAULT / OC_SW_LIMIT_MA	u16	1000 / 500	22000	SW hard / soft OC
0x72	FOC_KE_UV_S_RAD	u16	1	65000	λ — read by the 6-step ABS_FLOOR (§8)
0x56 / 0x57	ZC_DEMAG_DUTY / BLANK_EXTRA	u8	20 / 0	90 / 30	demag blanking
0x6C / 0x6D	MORPH_LOCK_ZC / TOL_PCT	u8	2 / 5	20 / 60	morph → CL lock gate

Live tuning. There is no `FEATURE_LIVE_TUNE` symbol; live tuning is state-gate exceptions in `HandleSetParam`: all `SET_PARAM` is IDLE-only *except* the AN1078 IDs `0x90–0x93` and the OC chop IDs `0x58/0x59` (which push straight to `HAL_CMP3_SetThreshold()`). With `FEATURE_GSP_EEPROM=0` in this dev build, all writes are RAM-only and revert to `profileDefaults[MOTOR_PROFILE]` on reset/reflash. A CRC-16 defaults-signature at EEPROM offset 100 auto-invalidates a stale overlay when a profile's compiled defaults change.

10.4 Telemetry snapshot

The 248-byte packed `GSP_SNAPSHOT_T` is captured tear-safely (8/16-bit atomic, 32-bit double-read, `stepPeriodHR` via the `writeSeq` seqlock; window peaks read-and-reset each capture so each frame is the most-recent ~20 ms envelope). Key fields (offsets):

Off	Field	Type	Meaning
0–3	state, faultCode, currentStep, direction	u8×4	FSM + sector
4 / 6	throttle / dutyPct	u16 / u8	0–2000 / duty-100/LOOPTIME_TCY
8 / 10 / 14 / 16	vbusRaw / ibusRaw / bemfRaw / zcThreshold	u16	ADC counts
18 / 20	stepPeriod / goodZcCount	u16	eRPM = 450000/stepPeriod (host)
30 / 31 / 40	hwzcEnabled / hwzcPhase / hwzcStepPeriodHR	u8/u8/u32	HR period (seqlock)
32 / 36	hwzcTotalZcCount / TotalMissCount	u32	measured vs guessed
52 / 56	clpciTripCount / fpciTripCount	u32	HW chop / fault PCI
170	hwzcNoiseReject	u32	ZCs rejected (off-time / interval gate)
198 / 200	ibusWinMax / Min	u16	bus current window (negative = freewheel artifact)
208–228	speedPi* block	mixed	parked outer-loop telemetry
228	cpuLoadPer mille	u16	main-loop CPU load vs motor-off baseline
230	hwzcMissBySector[6]	u16×6	per-sector guess tally (proved falling-polarity masking)
242 / 244	fallOffBemfMin / Max	u16	falling-sector OFF-centre envelope
246	ibusAvg	u16	IIR low-pass of ibusRaw (~5.7 ms TC) — last field

11 Consolidated Mathematics Reference

Every quantitative relationship in the firmware, in one place. Reference profile values shown where they make the formula concrete.

11.1 Clock, period & duty

```
FCY = 100 MHz · PWM_CLOCK_MHZ = 400 · SCCP_CLOCK_HZ = 108 (1 HR tick = 10 ns)
LOOPTIME_μs = 106/PWMFREQUENCY_HZ = 106/45000 = 22.222 μs
LOOPTIME_TCY = (22.222 × 8 × 400) - 16 ≈ 71 095 (MPER)
MAX_DUTY = LOOPTIME_TCY × 99/100 · D = duty / LOOPTIME_TCY
DEADTIME_COUNTS = (DEADTIME_NS/1000)×16×400 = 1920 · MIN_DUTY = 2×DEADTIME_COUNTS = 3840
```

11.2 Period ↔ eRPM (the 1e9 constant)

```
6 steps/erev ⇒ step-frequency = eRPM/10 Hz ⇒ stepPeriod = 10/eRPM s = (10/eRPM)·108 ticks
HWZC_TICKS_TO_ERPM(t) = HWZC_ERPM_TO_TICKS(e) = 109/x (self-inverse)
HWZC_ADC_TO_SCCP2(t) = t × (108/45000) ≈ t × 2222 · RT_HWZC_MIN_STEP_TICKS = 109/maxClosedLoopErpm
TIMER1_TO_ADC_TICKS(t) = t × PWMFREQUENCY_HZ/10000 = t × 4.5 (was ×12/5 – the 24 kHz bug, fixed)
```

11.3 ZC neutral threshold

```
rawThresh = vbusRaw × duty / ZC_DUTY_DIVISOR
ZC_DUTY_DIVISOR = 2·LOOPTIME_TCY (→ VdcD/2) | (26·LOOPTIME_TCY)/10 (→ VdcD/2.6, micros)
zcThreshSmooth += (rawThresh - zcThreshSmooth + 2) >> 2 (α = 1/4)
```

11.4 Filter-lag compensation (threshold bias)

```
HWZC_FILTER_K_Q15 = 2π·109·τ·32768/60 = 102 943 706 (τ = 30 μs)
omegaTauQ15 = K/stepP = (ωeτ)·32768, clamp ≤ HWZC_FILTER_MAX_OMEGA_Q15 = 24000 (≈ 0.73 rad)
amp = zcAmpForFilterComp (else thresh) · ampPct = 50 (rising and falling)
offset = ((amp × omegaTauQ15) >> 15) × ampPct/100, clamp ≤ HWZC_FILTER_MAX_OFFSET = 600
rising: CMPLO = thresh - offset + DEADBAND · falling: CMPLO = thresh + offset - DEADBAND
HWZC_CMP_DEADBAND = 2 (micros) / 4 (default)
```

11.5 Timing advance

```
advDeg = MIN + (MAX-MIN)·(eRPM - RAMP_TARGET)/(FULL - RAMP_TARGET), clamped [MIN, MAX]
MIN = TIMING_ADVANCE_MIN_DEG = 0° · MAX = timingAdvMaxDeg (20°) · FULL = RT_TIMING_ADV_FULL_ERPM
reactive: commDelay = stepPeriodHR·(30-advDeg)/60 · PI: setValue = ((30+advDeg)·T)/60
advFp8 = ((30+advDeg)·256)/60 ∈ [128,256]
```

11.6 Sector PI & IIR

```
capValue = lastCaptureHR - lastCommStamp · delta = capValue - setValue
clamp: high RPM +T/16 / -T/32; else ±T/8 · handoff-damp neg ≤ T>>6 (24 events)
Kp = 2ζωn = 0.25, Ki = ωn2 = 0.0625 (ωn=0.25, ζ=0.5)
integratorF += Ki·delta; T = integratorF + Kp·delta; both ≥ RT_HWZC_MIN_STEP_TICKS
reactive IIR: newPeriod = (3·stepPeriodHR + interval)/4 · defensive: T ×= 1.01/event
filter-comp period: stepPeriodForFilterComp via slow IIR (shift 11)
```

11.7 ABS_FLOOR no-load guard

```
Pff = 181.380 × λ / (vbus_v × dutyFrac) [HR ticks], λ = focKeUvSRad (μV·s/rad)
floorP = Pff × 100/HWZC_ABS_FLOOR_OVERSPEED_PCT (130), applied if dutyFrac > 0.03 ∧ vbus_v > 6
shrink-only: clamp T → floorP iff (T < floorP ∧ T < Tprev)
λ for a new motor: focKeUvSRad = round(5.5132×106/(KV·PP))
```

11.8 Current & voltage sensing

```
OC_TRIP_MV(ma) = 1650 + ma·3·2495/100000 · OC_MV_TO_COUNTS(mv) = mv·4096/3300
⇒ ≈ 93 counts/A, 2048 = 0 A · ibus_corrected = rawBus + (2048 - g_ocBiasAdc)
CMP3 DAC: v = dacVal - (2048 - g_ocBiasAdc) · auto-zero gate: 64 samples, spread ≤ 16
VBUS_SCALE_V_PER_COUNT = 3.3·23.2/4095 = 0.018696 V/count
SW soft OC: reduction = (ibus-OC_SW_LIMIT)·(duty-MIN_DUTY)/range
```

11.9 Blanking & gates

```
HW blank = period × HWZC_BLANKING_PERCENT/100 (14 %), min 1 μs
SW blank = stepPeriod × ZC_BLANKING_PERCENT/100 (3 %) + demag(D > zcDemagDutyThresh), cap stepPeriod/4
plausibility: reject interval < max(stepPeriodHR×HWZC_MIN_INTERVAL_PCT/100 (50 %), RT_HWZC_MIN_STEP_TICKS)
LEB = OC_LEB_BLANKING_NS/1000 × PWM_CLOCK_MHZ = 200/1000 × 400 = 80 counts
```

11.10 GSP framing

```
CRC16: crc=0xFFFF; crc ^= byte<<8; >8: crc = (crc&0x8000) ? (crc<<1)^0x1021 : crc<<1
over LEN+CMD_ID+PAYLOAD (START excluded), TX high-byte-first · host eRPM = 450000/stepPeriod
```

12 Appendix: Feature Flags & Glossary

12.1 Key feature flags (reference build)

Flag	State	Effect
FEATURE_ADC_CMP_ZC	1	Hardware comparator ZC path enabled
FEATURE_HWZC_SECTOR_PI	1	Autonomous-timer + PI architecture (vs reactive)
FEATURE_HWZC_PI_FLOAT	1	Float parallel-form PI (ω_n/ζ gains)
FEATURE_HWZC_PI_DEFENSIVE	1	Relax-when-silent self-heal
FEATURE_HWZC_FILTER_COMP	1	RC filter-lag threshold compensation (§6)
FEATURE_HWZC_FALLING_SW	1	Per-polarity: HW comparator rising-only, SW falling
FEATURE_HWZC_ABS_FLOOR	1	No-load period floor from λ (§8)
FEATURE_TIMING_ADVANCE	1	Speed-scheduled commutation advance (§7)
FEATURE_SINE_STARTUP	1	Sine V/f align → ramp → morph startup
FEATURE_OC_AUTOZERO	1	Bus-ADC rest-bias re-centering (§9.2)
FEATURE_VBUS_FAULT / REGEN_BRAKE / EMERGENCY_HOLD	1	Multi-tier bus-voltage protection (§9.3)
FEATURE_SPEED_PI	0	Outer eRPM loop — parked (§9.4)
FEATURE_AM32_STARTUP / IF_STARTUP / PLL_STARTUP	0	Alternate startup strategies (off for micros)
FEATURE_GSP / GSP_EEPROM	1 / 0	Host protocol on; persistence off (RAM-only tuning)

12.2 Glossary

Term	Meaning
ZC	Zero-crossing — the floating-phase BEMF crossing the virtual neutral; the rotor-position reference
HWZC / SW-ZC	Hardware (digital comparator) vs software (ADC-ISR threshold) zero-cross detection
HR tick	SCCP2 high-resolution timestamp tick = 10 ns (100 MHz)
stepPeriodHR / timerPeriod	Measured / PI-controlled commutation (60° sector) period in HR ticks
eRPM	Electrical RPM = mechanical RPM × pole-pairs
MORPH	The sine → trapezoid handoff state that blends open-loop drive into closed-loop ZC commutation
λ (focKeUvSRad)	Motor flux/back-EMF constant, $\mu\text{V}\cdot\text{s}/\text{rad}$ electrical; read by the 6-step ABS_FLOOR guard
advance	6-step commutation phase lead (NOT field weakening) compensating inductive + detection lag
chop (CLPCI)	Cycle-by-cycle hardware current limit: CMP3 truncates each PWM pulse via the PWM current-limit PCI
rej%	Fraction of comparator fires rejected ($\approx$ PWM-OFF fraction) — not a guess rate
GSP	Garuda Serial Protocol — the UART command/telemetry API

Document scope & provenance. Generated from a complete read of `dspic33AKESC/` on branch `ak512-port`, reference build `MOTOR_PROFILE_6` (VEX 4000 KV). Constants and citations reflect the source as committed; values in `gspParams` are runtime-tunable and may differ per active profile. Stale in-code comments referencing a "24 kHz ADC ISR" are superseded by `PWMFREQUENCY_HZ = 45000` throughout.

13 Worked Example — One Commutation Cycle at 100 000 eRPM

To make the timing chain concrete, trace a single sector with the reference profile (45 kHz PWM, `maxClosedLoopErpm` = 240 000, `timingAdvMaxDeg` = 20°, `RAMP_TARGET_ERPM` = 3000) running steadily at 100 000 eRPM, duty ≈ 38 %, $V_{bus} \approx 10$ V.

Step A — period in HR ticks

```
stepPeriodHR = 109/eRPM = 109/100 000 = 10 000 ticks = 100 µs per 60° sector
one electrical revolution = 6 × 100 µs = 600 µs ⇒ 1 667 erev/s
```

At 45 kHz the ADC ISR runs every 22.22 µs, so a 100 µs sector spans only ≈ **4.5 ADC samples** — already coarse for the software path, which is why HWZC (1 MHz comparator) owns detection here.

Step B — advance angle

```
advDeg = 0 + (20-0)·(100 000 - 3 000)/(240 000 - 3 000) = 20·(97 000/237 000) = 8.18° → 8°
advFp8 = ((30+8)·256)/60 = 9728/60 = 162 (Q8, = 0.633)
```

Step C — setpoint & commutation instant

```
setValue = (advFp8·T)≫8 = (162·10 000)/256 = 6328 ticks (= 63.3 % into the sector)
reactive commDelay = T·(30-advDeg)/60 = 10 000·(22/60) = 3667 ticks after the ZC
```

The ZC ideally lands at the sector midpoint (5000 ticks / 30°). With 8° advance the loop holds the ZC at 6328 ticks past the *previous* commutation, so commutation fires 8° before the next rotor sector boundary — covering the inductive + detection lag.

Step D — filter-lag compensation bias

```
omegaTauQ15 = K/stepP = 102 943 706 / 10 000 = 10 294 (=  $\omega_e \tau = 0.314$  rad = 18°)
 $\omega_e = 2\pi \cdot (100 000/60) = 10 472$  rad/s ⇒  $\omega_e \tau = 10 472 \cdot 30 \mu s = 0.314$  ✓ (matches)
assume amp (zcAmpForFilterComp) ≈ 220 counts ⇒ offset = (220·10 294 >> 15)·50/100 = (69)·0.5 = 35 counts
rising Cmpl0 = zcThreshold - 35 + 4(deadband) ; falling Cmpl0 = zcThreshold + 35 - 4
```

So at 100 k the comparator threshold is pulled ≈ 35 counts toward the oncoming signal to cancel an 18° RC lag — the single largest dynamic correction in the loop, and the reason ZC stays centred rather than walking late with speed.

Step E — blanking & plausibility window

```
blankTicks = 10 000·14/100 = 1400 ticks (14 µs) masked after each commutation
plausibility floor = max(10 000·50/100, RT_HWZC_MIN_STEP_TICKS=4167) = 5000 ticks — any interval < 5000 rejected
```

Step F — PI correction for a 200-tick early capture

```
say capValue = 6128 (200 ticks early) ⇒ delta = 6128 - 6328 = -200
clamp (mid RPM, T > 6667-tick HIGH threshold ⇒ symmetric ±T/8 = ±1250): -200 passes
integratorF += 0.0625·(-200) = -12.5 ⇒ Tnew = integratorF + 0.25·(-200) = T - 12.5 - 50 ≈ 9938 ticks
new commanded eRPM = 109/9938 = 100 624 — a 0.6 % nudge faster to recentre the ZC
```

The tight negative clamp (here T/8, tightening to T/32 above 150 k) prevents a string of early captures from ratcheting the period down onto a half-period phantom; the ABS_FLOOR guard (§8) is the absolute backstop if it ever tried.

14 Peripheral Register Configuration Reference

14.1 PWM generators (PG1/2/3)

Register / field	Value	Meaning
PG1CON.MODSEL	4	Center-aligned PWM
PGxIOCON1.PMOD	0	Complementary HS/LS with hardware deadtime
PGxDTH / PGxDTL	1920	Deadtime high/low = 300 ns
MPER	71 095	Master period = LOOPTIME_TCY
PG1EVT1.ADTR1EN1	1	PG1TRIGA drives ADC trigger 1
PG1TRIGA (CAHALF/TRIGA)	0 / MPER/2	ADC sample at freewheel OFF-centre
PGxIOCON2 (override word)	0 / 0x301000 / 0x300000	PWM_ACTIVE / LOW / FLOAT
PGxCLPCI2	0x40000000	PCI source = CMP3 output (bit30)
PGxCLPCI1.AQSS / ACP	0b000 / 0b011	forced-accept, latched chop
PGxLEB	80	Leading-edge blanking = 200 ns

14.2 ADC channel map (AK512)

Signal	Channel / pin	Trigger	Sample
Phase B BEMF (ISR src)	AD1CH3 / RA10	PG1TRIGA	OFF-centre, SAMC=5
Phase A BEMF	AD1CH4 / RA9	PG1TRIGA	OFF-centre
Phase C BEMF	AD2CH3 / RB8	PG1TRIGA	OFF-centre
HWZC comparator (active float)	AD1CH1/2, AD2CH2	SCCP3 (TRG=34)	1 MHz, SAMC=3 (~205 ns)
Vbus	AD3CH2 / RF0	PG1TRIGA	OFF-centre
Ibus	AD3CH1 / RA5 (OA3OUT)	PG1TRIGA	SAMC=3
Ia / Ib monitor	AD1CH0 / AD2CH0 (OA1/2OUT)	PG1TRIGA	mid-ON, SAMC=3
Pot (throttle)	AD2CH1 / RB15	PG1TRIGA	—

Only the **active floating phase's** HWZC channel is triggered (`TRG1SRC=34`); the other two are gated off (`TRG1SRC=0`). Two simultaneous 1 MHz burst channels on one AD core starve the PWM-triggered control channel — a bench-proven constraint, and why `HAL_ADC_BemfBurstOff()` must run on HWZC shutdown.

14.3 Comparator / DAC (CMP3 OC chop)

Field	Value	Meaning
DAC3CMP.INPSEL	3	CMP input D = OA3 output (the MC510 fix; INPSEL=0 watched the wrong node)
DAC3CMP.CMPPOL	0	Output HIGH when OA3OUT > DAC
DAC3CMP.HYSSEL	0b11	45 mV hysteresis (max)
DAC3CMP.FLTREN	1	Digital filter on
DAC3DAT.DACDAT	OC_CMP3*_DAC	Threshold, shifted by HAL_CMP3_SetThreshold for auto-zero
DAC trim source	flash 0x7F20E0	Factory INL/DNL (wrong address left CMP3 reference dead)

14.4 SCCP timers

Timer	Mode	Clock	Role
SCCP1	32-bit one-shot	100 MHz	Blanking expiry & next-commutation deadline
SCCP2	32-bit free-run, PR=0xFFFFFFFF	100 MHz	10 ns ZC timestamp / period clock
SCCP3	32-bit periodic, PR=100	100 MHz	1 MHz trigger for the HWZC comparator channel
SCCP4	32-bit input capture	100 MHz	RX servo-PWM / DShot edge capture (not in motor loop)
Timer1	1:8 prescale	F _{CY} /8	100 µs system tick — arm/align/ramp/recovery counters

15 Complete Parameter Descriptor Table (58 entries)

Every settable parameter, in enumeration order, with type, group, bounds and units. Groups: 0 STARTUP, 1 CLOSED_LOOP, 2 OVERCURRENT, 3 ZC_DETECT, 4 DUTY_SLEW, 5 VOLTAGE, 6 RECOVERY, 7 MOTOR_HW, 8 FOC_MOTOR, 9 FOC_TUNING, 10 FOC_STARTUP, 11 AN1078.

ID	Name	T	Grp	Min	Max	Units
0x15	RAMP_TARGET_ERPM	u16	0	500	20000	eRPM
0x16	RAMP_ACCEL_ERPM_PER_S	u16	0	50	20000	eRPM/s
0x17	RAMP_DUTY_PCT	u8	0	5	80	%
0x51	ALIGN_DUTY_PCT	u8	0	2	50	%
0x52	INITIAL_ERPM	u16	0	50	1000	eRPM
0x54	SINE_ALIGN_MOD_PCT	u8	0	2	50	%
0x55	SINE_RAMP_MOD_PCT	u8	0	5	80	%
0x20	CL_IDLE_DUTY_PCT	u8	1	0	30	%
0x22	TIMING_ADV_MAX_DEG	u8	1	0	45	°
0x30	HWZC_CROSSOVER_ERPM	u16	1	500	20000	eRPM
0x53	MAX_CL_ERPM	u32	1	5000	350000	eRPM
0x56	ZC_DEMAG_DUTY_THRESH	u8	1	20	90	%
0x57	ZC_DEMAG_BLANK_EXTRA	u8	1	0	30	%
0x42	OC_SW_LIMIT_MA	u16	2	500	22000	mA
0x41	OC_FAULT_MA	u16	2	1000	22000	mA
0x58	OC_LIMIT_MA	u16	2	501	22000	mA
0x59	OC_STARTUP_MA	u16	2	5000	22000	mA
0x5A	RAMP_CURRENT_GATE_MA	u16	2	0	22000	mA
0x64	ZC_BLANKING_PCT	u8	3	1	15	%
0x65	ZC_ADC_DEADBAND	u8	3	0	20	counts
0x66	ZC_SYNC_THRESHOLD	u8	3	4	20	—
0x67	ZC_FILTER_THRESHOLD	u8	3	1	10	—
0x60	DUTY_SLEW_UP	u8	4	1	20	%/ms
0x61	DUTY_SLEW_DOWN	u8	4	1	50	%/ms
0x62	POST_SYNC_SETTLE_MS	u16	4	100	5000	ms
0x63	POST_SYNC_SLEW_DIV	u8	4	1	16	—
0x68	VBUS_OV_ADC	u16	5	2000	4000	counts
0x69	VBUS_UV_ADC	u16	5	200	2000	counts
0x6A	DESYNC_COAST_MS	u16	6	50	1000	ms
0x6B	DESYNC_MAX_RESTARTS	u8	6	0	10	—
0x6C	MORPH_LOCK_ZC_COUNT	u8	0	2	20	—
0x6D	MORPH_LOCK_TOL_PCT	u8	0	5	60	%
0x6E	IF_CURRENT_CA	u16	0	100	2000	cA
0x6F	IF_RAMP_ERPM_PER_S	u16	0	1000	60000	eRPM/s
0x50	MOTOR_POLE_PAIRS	u8	7	1	20	—
0x70	FOC_RS_MOHM	u16	8	10	10000	mΩ
0x71	FOC_LS_UH	u16	8	1	10000	μH
0x72	FOC_KE_UV_S_RAD	u16	8	1	65000	μV·s/rad (λ)
0x73	FOC_VBUS_NOM_CV	u16	8	500	6000	cV
0x74	FOC_MAX_CURRENT_CA	u16	8	50	5000	cA
0x75	FOC_MAX_ELEC_RAD_S	u16	8	500	30000	rad/s
0x76	FOC_KP_DQ_MILLI	u16	9	1	60000	×1e-3

0x77	FOC_KI_DQ	u16	9	1	60000	—
0x78	FOC_OBS_LPF_MILLI	u16	9	10	900	×1e-3
0x79–0x80	FOC_ALIGN/RAMP_IQ, TIMES, RATE, HANDOFF, FAULTS	u16	10	—	—	FOC startup set
0x90	AN1078_THETA_BASE_DEGX10	u16	11	0	3600	deg×10 (live)
0x91	AN1078_THETA_K_E7	u16	11	0	1000	×1e-7 (live)
0x92	AN1078_KSLIDE_MV	u16	11	100	30000	mV (live)
0x93	AN1078_ID_FW_MAX_DECIA	u16	11	0	200	dA (live)

16 Design Rationale & Hard-Won Lessons

Several non-obvious design choices are the residue of measured failures. They are collected here because they are the parts most likely to be "fixed" incorrectly by a future maintainer.

Choice	Naive alternative	Why the alternative fails
OFF-centre ADC sample (MPER/2)	Pulse-centre sample	Pulse-centre lands in driven coupling → noisy; capped at 174 k + OC vs 215 k at MPER/2
Filter-comp amplitude = measured BEMF	Use <code>zcThreshold</code> ($\propto$ duty)	Duty-dependent offset → regen positive feedback → ZC walks / desync
Slow IIR for filter-comp period	Live <code>timerPeriod</code>	Shrink → bigger $\omega\tau$ → earlier fire → shrink: runaway
Asymmetric PI clamp (neg tighter)	Symmetric clamp	Negative drift commutates ahead of rotor at ceiling → 22 A → UV
HW comparator rising-only + SW falling	Comparator both polarities	PWM-ON coupling masks the falling crossing → 100 % odd-sector misses
ABS_FLOOR shrink-only	Always clamp to floor	Would block a legitimate high-speed coast-down through the floor
λ per motor (focKeUvSRad)	One λ for all	Wrong λ pins the motor at 130 % ($\lambda_{\text{true}}/\lambda_{\text{set}}$) of no-load (the VEX 51 % symptom)
3-sample OC fault debounce	Single-sample trip	One rail-spiking bus sample latched phantom OC with the bridge off
OC auto-zero quiescence gate (spread ≤ 16)	Always latch the bias	Re-arm with coasting rotor latches a ~ 7 A-wrong bias from regen ripple
Emergency-hold freezes slew-down only	Hard brake on OV	Braking dumps more regen → 47.8 V PSU-OV cascade; freezing breaks the loop
TIMER1_TO_ADC_TICKS from PWMFREQUENCY_HZ	Hardcoded $\times 12/5$	The 24 → 45 kHz migration left every seeded period $1.875\times$ too fast ("5696 eRPM" handoff)

The single most important invariant: the firmware never commands the rotor *faster* than it can physically be — not via the PI (asymmetric clamp), not via harmonic lock (ABS_FLOOR), not via the speed map (period floor). Every speed-related guard is one-directional: it relaxes toward the measured rotor and refuses to push ahead of it. That is what makes a sensorless 6-step loop survive the BEMF ceiling instead of detonating into a wrong-angle current spike.

17 Complete Telemetry Snapshot Map (248 bytes)

The full packed `GSP_SNAPSHOT_T` layout. Float (FOC) fields are zero in pure-6-step builds; the 6-step phase-current block at 174+ is populated only when no FOC feature is enabled. Window-peak fields are read-and-reset each capture.

Off	Field	Type	Meaning
0	state	u8	ESC_STATE_T
1	faultCode	u8	FAULT_CODE_T
2	currentStep	u8	commutation sector 0–5
3	direction	u8	rotation direction
4	throttle	u16	0–2000
6	dutyPct	u8	duty·100/LOOPTIME_TCY
8	vbusRaw	u16	bus voltage ADC counts
10	ibusRaw	u16	bus current (2048 bias, ~93 cnt/A)
12	ibusMax	u16	bus current all-run peak
14	bemfRaw	u16	floating-phase BEMF sample
16	zcThreshold	u16	ZC neutral threshold
18	stepPeriod	u16	T1-tick period; host eRPM=45000/stepPeriod
20	goodZcCount	u16	consecutive good ZCs
22 / 23 / 24	risingZcWorks / fallingZcWorks / zcSynced	u8	lock flags
26 / 28	zcConfirmedCount / zcTimeoutForceCount	u16	confirmed / forced commutations
30 / 31	hwzcEnabled / hwzcPhase	u8	HWZC enable + watch phase
32 / 36	hwzcTotalZcCount / TotalMissCount	u32	cumulative measured / missed
40	hwzcStepPeriodHR	u32	HR commutation period (seqlock)
44	hwzcDbgLatchDisable	u8	diag
46 / 47 / 48 / 50	morphSubPhase / morphStep / morphZcCount / morphAlpha	mixed	morph blend state & α
52 / 56	clpciTripCount / fpciTripCount	u32	CMP3 chop / fault-PCI trips
60 / 64	systemTick / uptimeSec	u32	free-running tick / seconds
68–148	foc* block (Id, Iq, theta, omega, Vbus, Ia/Ib, observer, flux, λ_{est} , PI integrators, modIndex, confidence, offsets)	f32/...	FOC telemetry — zero in 6-step
150–169	smoResidual, pllInnovation, pllOmega, omegaOI, handoffCtr, smoObservable	f32/...	observer diagnostics (FOC)
170	hwzcNoiseReject	u32	ZCs rejected (off-time/interval gate)
174 / 176	iaRaw / ibRaw	u16	instantaneous phase A/B current
178–184	iaMax/iaMin/ibMax/ibMin	u16	phase-current window envelope
186–196	ia/Ib *AtFault, *MaxAtFault, *MinAtFault	u16	frozen at first PCI
198 / 200	ibusWinMax / ibusWinMin	u16	bus current window (neg = freewheel artifact)
202–206	ibusAtFault / MaxAtFault / MinAtFault	u16	frozen at fault
208 / 210	speedPiEnabled / ZcsSinceEnable	u8/u16	outer-loop state
212 / 216 / 220 / 224	speedPiTarget / LastError / OutputDuty / IntegratorF	u32/i32/u32/f32	parked speed-PI telemetry
228	cpuLoadPer mille	u16	main-loop CPU load vs motor-off baseline
230	hwzcMissBySector[6]	u16×6	per-sector guess tally
242 / 244	fallOffBemfMin / Max	u16	falling-sector OFF-centre envelope
246	ibusAvg	u16	IIR low-pass of ibusRaw (~5.7 ms TC) — last field

18 Motor Profile Comparison

The firmware ships nine built-in profiles in `profileDefaults[]`, selected at compile time by `MOTOR_PROFILE` (direct index). The key per-profile timing/detection values illustrate how the same algorithms re-target across very different motors — note especially how the high-KV/low-inductance micros (6/7/8) push the HWZC crossover up and the falling-SW cap down.

#	Profile	Motor	PP	HWZC cross.	Falling-SW cap	max CL eRPM
0	HURST	Hurst DMB2424, high-R	5	1500	70 000 (default)	20 000
1	A2212	A2212 1400 KV, low-R	7	1500	35 000	120 000
2	5010 (=2810)	2810 1350 KV 12N14P	7	1500	70 000	260 000
3	5055	5055-class	—	—	—	80 000
4	COBRA	Cobra CM-2814/36 470 KV	7	—	—	—
5	XROTOR	XRotor 3110 1150 KV	7	—	—	210 000
6	VEX (reference)	VEX 14 mm 4000 KV micro	6	24 000	45 000	240 000
7	1407_2S	1407 4000 KV 9N12P @ 2S	6	6 000	50 000	202 000
8	1407_3S	1407 4000 KV 9N12P @ 3S	6	6 000	50 000	260 000

Three per-profile compile-time relaxations apply only to the high-KV micros (profiles 6/7/8), where the 10 V BEMF is only a few ADC counts above quantisation: the ZC neutral divisor (2.6 vs 2.0), `FEATURE_AM32_STARTUP` off (forced sine ramp instead of the phantom-prone kick), `FEATURE_HWZC_VERIFY_READS` off, and `HWZC_CMP_DEADBAND` 2 vs 4. Everything else — the 2810 and the larger motors — keeps the default, more conservative values. Profile selection therefore changes *tuning*, never the algorithm.

18.1 Bringing up a new motor — checklist

1. **λ first.** Set `focKvUvSRad = round(5.5132×106/(KV·PP))`. A wrong λ pins the motor at a clean fraction of its real speed (§8) — diagnose this before touching anything else.
2. **Pole pairs.** `motorPolePairs` = magnet poles ÷ 2.
3. **Speed cap.** `maxClosedLoopErpm` = nominal no-load (KV·V·PP) is a safe default; this is also the advance anchor and the period floor. Live-tune `0x11` upward to explore field-... (*commutation-advance*)-limited headroom.
4. **Startup.** Port a proven low-hand-off startup (`rampTarget` ≈ 3000, `align/sineMod` low) before tuning the run loop.
5. **OC chain.** Scale `ocLimitMa/ocSwLimitMa/ocFaultMa` to the motor's stall current — the 24 V-class thresholds will never protect a 7 A micro.
6. **Crossover & falling cap.** High-KV/low-L motors: raise `hwzcCrossoverErpm` (engage HWZC only where BEMF is solid) and lower `HWZC_FALLING_SW_MAX_ERPM` (coast falling before the mid-band oscillation builds).

19 Runtime Parameter Struct (`GSP_PARAMS_T`)

The full runtime parameter aggregate, integer-scaled (no floats on the wire). `gspParams` is the live copy mutated by `SET_PARAM`; `gspDerived` holds the ISR-facing precomputed forms (duty caps in PWM counts, periods in ticks, OC ADC thresholds, CMP3 DAC values) refreshed by `GSP_RecomputeDerived()` after every change.

Field	T	Units	Role
rampTargetErpm / rampAccelErpmPerS / rampDutyPct	u16/u16/u8	eRPM, eRPM/s, %	open-loop ramp shape
clIdleDutyPct	u8	%	CL idle duty floor (0 $\Rightarrow$ MIN_DUTY)
timingAdvMaxDeg	u8	°	max commutation advance
hwzcCrossoverErpm	u16	eRPM	HWZC enable speed
ocSwLimitMa / ocFaultMa / ocLimitMa / ocStartupMa	u16	mA	OC ladder (soft/hard/chop-run/chop-start)
motorPolePairs	u8	—	pole pairs
alignDutyPct / initialErpm	u8/u16	%, eRPM	align duty, first OL step
maxClosedLoopErpm	u32	eRPM	period-floor clamp + advance anchor
sineAlignModPct / sineRampModPct	u8	%	sine-startup modulation depth
zcDemagDutyThresh / zcDemagBlankExtraPct	u8	%	demag-blanking schedule
rampCurrentGateMa	u16	mA	OL accel current gate (0=off)
dutySlewUpPctPerMs / dutySlewDownPctPerMs	u8	%/ms	duty slew limits
postSyncSettleMs / postSyncSlewDivisor	u16/u8	ms, —	post-sync settle window
zcBlankingPercent / zcAdcDeadband	u8	%, counts	SW-ZC blanking & deadband
zcSyncThreshold / zcFilterThreshold	u8	—	sync / noise-filter counts
morphLockZcCount / morphLockTolPct	u8	—, %	morph $\rightarrow$ CL lock gate
vbusOvAdc / vbusUvAdc	u16	counts	OV / UV thresholds
desyncCoastMs / desyncMaxRestarts	u16/u8	ms, —	recovery coast & retry cap
focRsMilliOhm / focLsMicroH / focKeUvSRad	u16	m Ω , μ H, μ V·s/rad	motor model (λ read by 6-step ABS_FLOOR)
focVbusNomCentiV / focMaxCurrentCentiA / focMaxElecRadS	u16	cV, cA, rad/s	FOC limits
focKpDqMilli / focKiDq / focObsLpfAlphaMilli	u16	—	FOC current-loop & observer gains
focAlign/RamplqCentiA, focAlign/IqRampTimeMs, focRampRateRps2, focHandoffRadS	u16	cA, ms, rad/s ²	FOC startup
focFaultOcCentiA / focFaultStallDeciRadS	u16	cA, drad/s	FOC fault limits
an1078ThetaBaseDegX10 / ThetaKE7 / KslideMv / IdFwMaxDecia	u16	deg $\times$ 10, $\times$ 1e7, mV, dA	SMC observer (live-tunable)
ifCurrentCa / ifRampErpmPerS	u16	cA, eRPM/s	I-f startup

Cross-validation invariants enforced on every `SET_PARAM` so the host cannot half-update an ordered pair: `initialErpm < rampTargetErpm < maxClosedLoopErpm`; `zcFilterThreshold < zcSyncThreshold`; `ocSwLimitMa < ocLimitMa  $\leq$  ocFaultMa`; `ocStartupMa  $\geq$  ocLimitMa`; `rampCurrentGateMa < ocLimitMa` (if non-zero); `vbusUvAdc < vbusOvAdc`.

End of application note. 19 sections · 18 tables · 10 figures. Every constant, register, formula and API herein is taken directly from the `dsPIC33AKESC/` source tree (branch `ak512-port`). For the firmware itself, the canonical references are `motor/hwzc.c` (ZC + PLL), `garuda_service.c` (ISR + state machine), `gsp/` (host protocol), and `hal/` (peripherals).